Chronos

A production-inspired distributed job scheduler

Intern Assignment Submission

Gokul Shiva

gokulshiva085@gmail.com

Live dashboard: `https://frontend-production-e4f9.up.railway.app`

Live API (Swagger): `https://api-production-f587.up.railway.app/docs`

Source code: `https://github.com/Shiva085000/chronos-scheduler`

Contents: overview & requirements coverage · dashboard screenshots · API reference · engineering design document · measured benchmarks · protocol & ER diagrams

Submission overview

Chronos is a production-inspired distributed job scheduling platform: an authenticated REST API accepts immediate, delayed, recurring (cron), batch and dependency-chained jobs into configurable queues (organization → project → queue), and a horizontally scalable worker fleet executes them with explicit reliability semantics — atomic `FOR UPDATE SKIP LOCKED` claiming, lease-based failure detection with heartbeats, fixed/linear/exponential retries with jitter, a dead letter queue, idempotent enqueue, and graceful drain on shutdown. PostgreSQL is the single source of truth; Redis only shaves claim latency (advisory wake channel) and dashboard load.

Highlights an evaluator may want first:

- **Reliability, verified:** `SIGKILL` a worker mid-job and the job is re-queued by the reaper within ~35 s with the attempt recorded as *lost* (§12 of the design document reproduces this and five other failure drills). 39 automated tests include claim-atomicity under 8 racing claimers and exact concurrency-cap enforcement.
- **Measured performance:** 309 → 1,505 jobs/s scaling 1 → 8 workers, 35 ms p50 dispatch latency (benchmarks section).
- **Every decision defended:** the design document explains each tradeoff (why Postgres over a broker, why leases over liveness pings, why at-least-once, why the DLQ is a status and not a table…).
- **Bonus features implemented:** workflow dependencies, queue sharding, RBAC (owner/admin/member/viewer), AI failure summaries, distributed locking (advisory locks), event-driven execution (wake channel), and login rate limiting.

Live demo & source

What	Where
Live dashboard	https://frontend-production-e4f9.up.railway.app — click **Demo Login** (`demo@example.com` / `demo12345`)
Live API (Swagger)	https://api-production-f587.up.railway.app/docs
Source code	https://github.com/Shiva085000/chronos-scheduler
Read-only RBAC user	`viewer@example.com` / `viewer12345` — mutating actions return 403
Prometheus metrics	https://api-production-f587.up.railway.app/metrics

The hosted stack runs on Railway: managed Postgres + Redis, the API, and a worker service (private networking between them; the workers claim from all demo queues). The dedicated-shard worker topology is exercised in the local compose stack and the automated tests.

Running locally

```
git clone https://github.com/Shiva085000/chronos-scheduler
cd chronos-scheduler
docker compose up --build -d      # postgres, redis, api, 2 workers, 1 sharded worker, frontend
docker compose exec api python -m app.scripts.seed   # demo data
```

Dashboard at http://localhost:3000, Swagger at http://localhost:8000/docs. Full test suite: `make test` (39 tests, unit + DB integration).

Within ~2 minutes of seeding, the dashboard shows: an instant success, a job holding a lease, the retry pipeline succeeding on attempt 3, a DLQ entry, a capped queue executing an atomic batch strictly one-at-a-time, an A → B → C workflow unlocking in order, a cron schedule firing every other minute, and a sharded queue served only by the shard worker.

Requirements coverage

Core requirements

Requirement	Status	Evidence
Authentication & project management; projects own queues	✅	JWT auth; org → project → queue chain provisioned at registration; `/projects`, `/queues` APIs
Queue configuration: priority, concurrency limits, retry policy, pause/resume, statistics	✅	queue rows carry all controls; caps enforced **atomically** under the queue row lock; per-queue stats endpoint + UI
Immediate, delayed, scheduled, recurring (cron), batch jobs via REST	✅	`run_at` for delayed/scheduled; `schedules` + advisory-locked materializer for cron (exactly-once per tick, missed ticks collapse); `POST /jobs/batch` all-or-nothing
Worker service: polls, atomic claims, concurrent execution, heartbeats, graceful shutdown	✅	asyncio worker; `SKIP LOCKED` claim CTE; one-transaction heartbeat extends liveness + all leases; SIGTERM drain releases unfinished jobs with the attempt refunded
Full lifecycle with retries and DLQ	✅	PENDING → RUNNING → SUCCEEDED / retry / DEAD; every transition a guarded UPDATE; DLQ requeue from UI/API
Retry strategies: fixed, linear, exponential	✅	per-job `backoff_strategy` + base/factor/cap/jitter, inheritable from queue defaults
Execution logs, retry history, worker assignment, timestamps, metrics	✅	append-only `job_attempts` audit trail; structured JSON logs with correlation ids; Prometheus metrics
Dashboard: queues, jobs, workers, retry failed, throughput & health	✅	screenshots below; 4 s polling for live updates

Bonus features

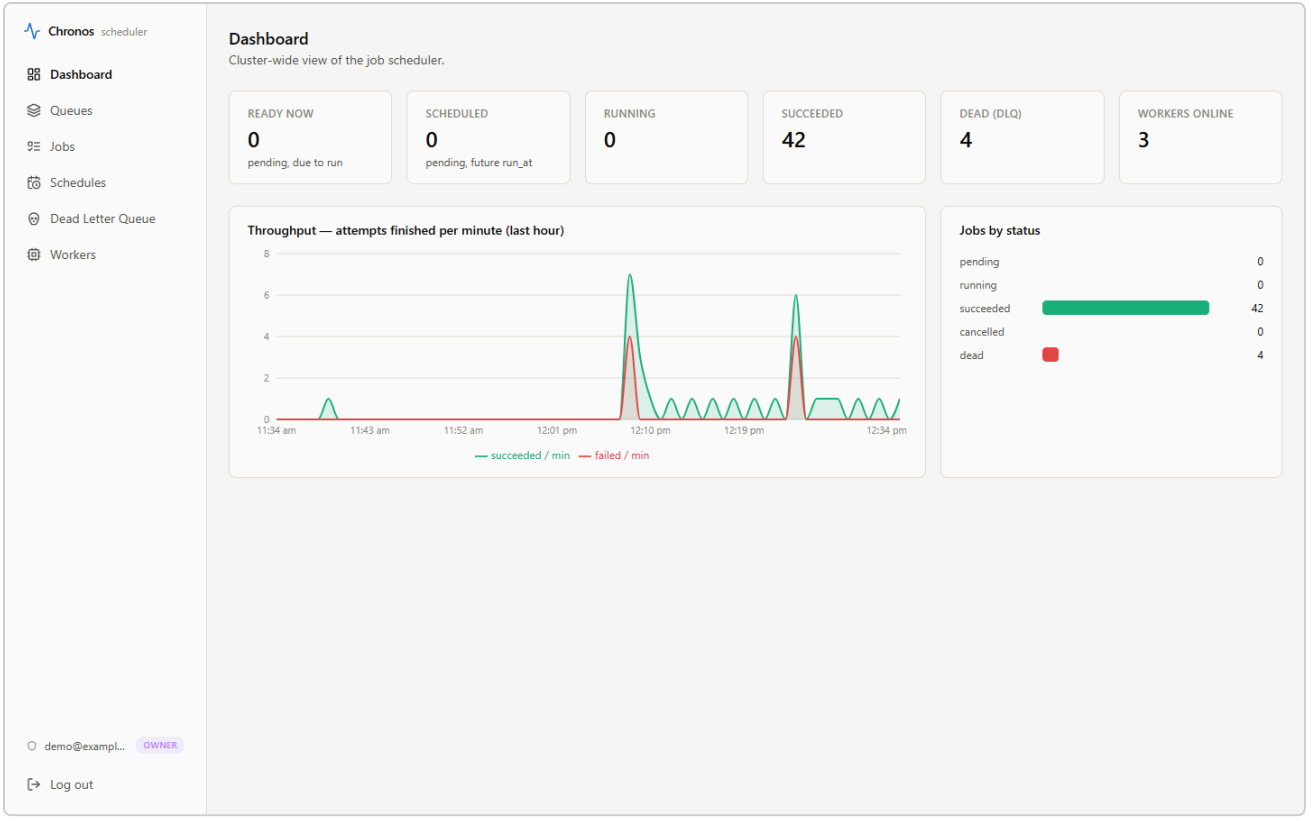
Bonus	Status	Notes
Workflow dependencies	✅	`job_dependencies`; claim scan admits a job only after all prerequisites SUCCEED
Rate limiting	✅ (login)	per-IP login throttle (Redis fixed window, fails open)
Distributed locking	✅	Postgres advisory locks coordinate the reaper and cron materializer cluster-wide
Queue sharding	✅	queues carry `shard_key`; a worker started with `WORKER_SHARD` claims only its shard (dedicated compose node)
Event-driven execution	✅	Redis pub/sub wake channel; poll fallback keeps correctness when Redis is down
WebSocket live updates	—	polling chosen (explicitly permitted); tradeoff documented
Role-based access control	✅	owner > admin > member > viewer, enforced per endpoint
AI-generated failure summaries	✅	Gemini root-cause notes on DLQ jobs, cached by error hash, degrades gracefully without an API key

Deliverables

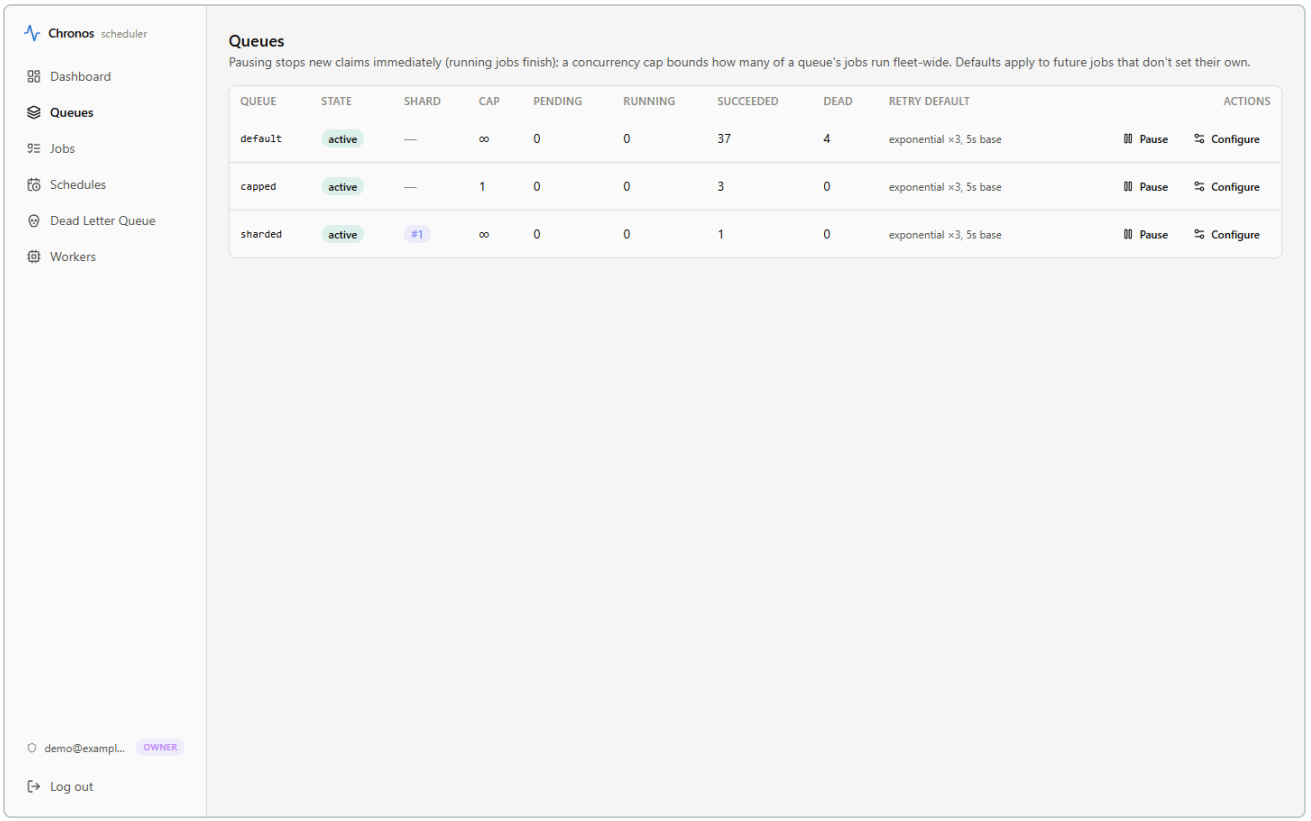
Deliverable	Where
Source code + setup	repository, `README.md` (one-command compose)
Architecture diagram	Protocol Diagrams §1 (this document)
ER diagram	Protocol Diagrams §11 (this document)
API documentation	generated reference below + live Swagger at `/docs`
Design decisions document	full engineering design document included below
Automated tests	39 tests: retry/cron domain units, claim atomicity, cap enforcement, batches, schedule materialization, registration, guardrails, worker resurrection

Dashboard walkthrough

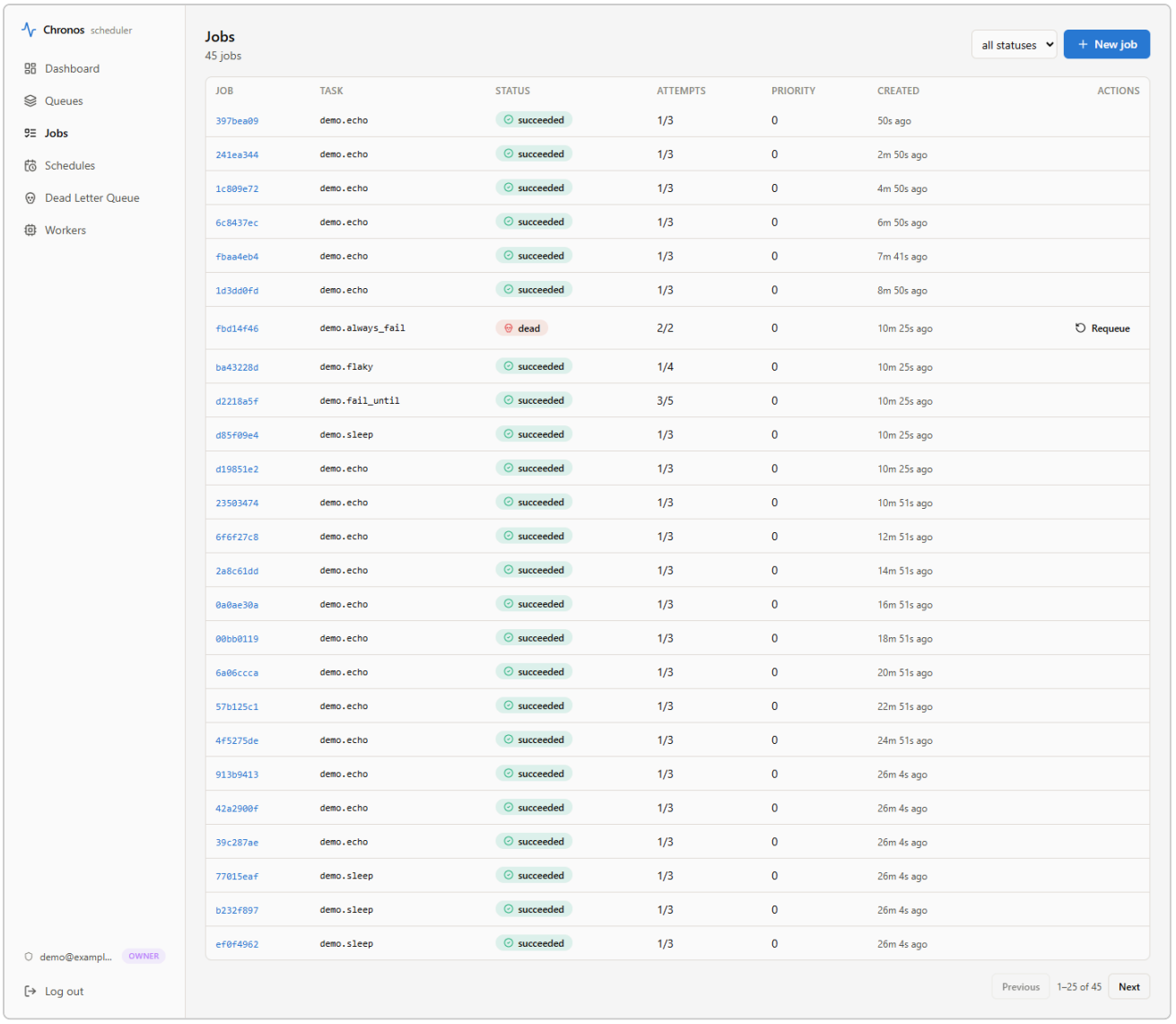
Captured from the running system with seeded demo data.



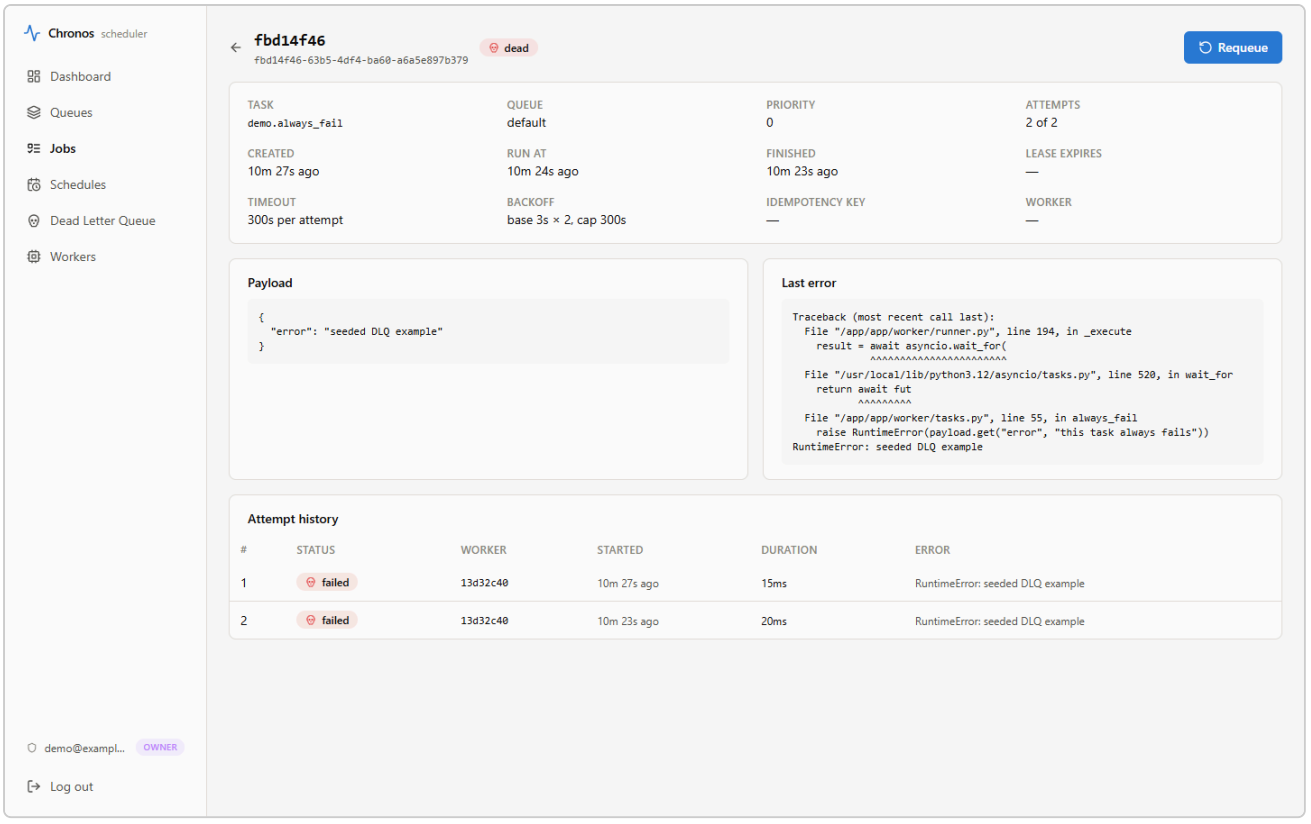
Dashboard — cluster stats, throughput (succeeded/failed per minute), status distribution; polls live.



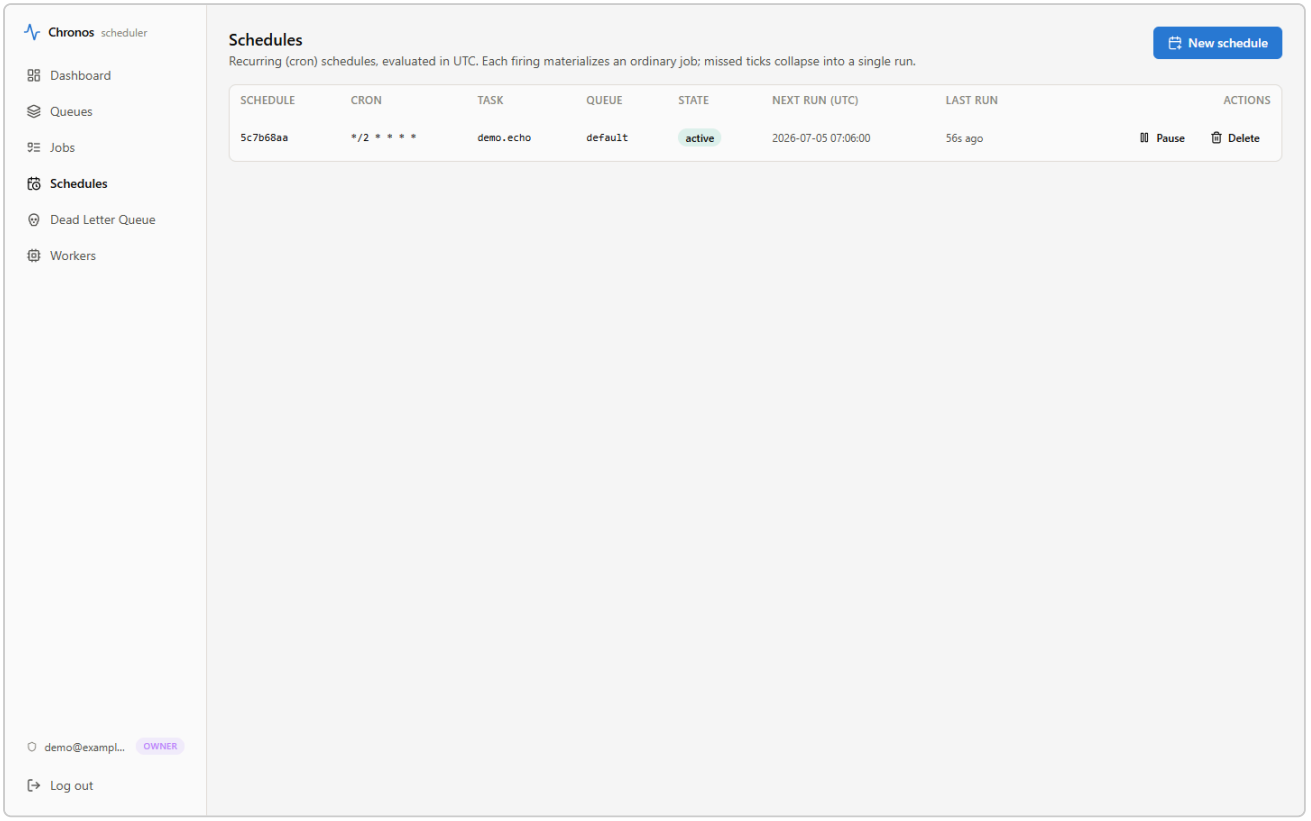
Queues — per-queue counts, pause/resume, shard badges, concurrency caps, default retry policy, config dialog.



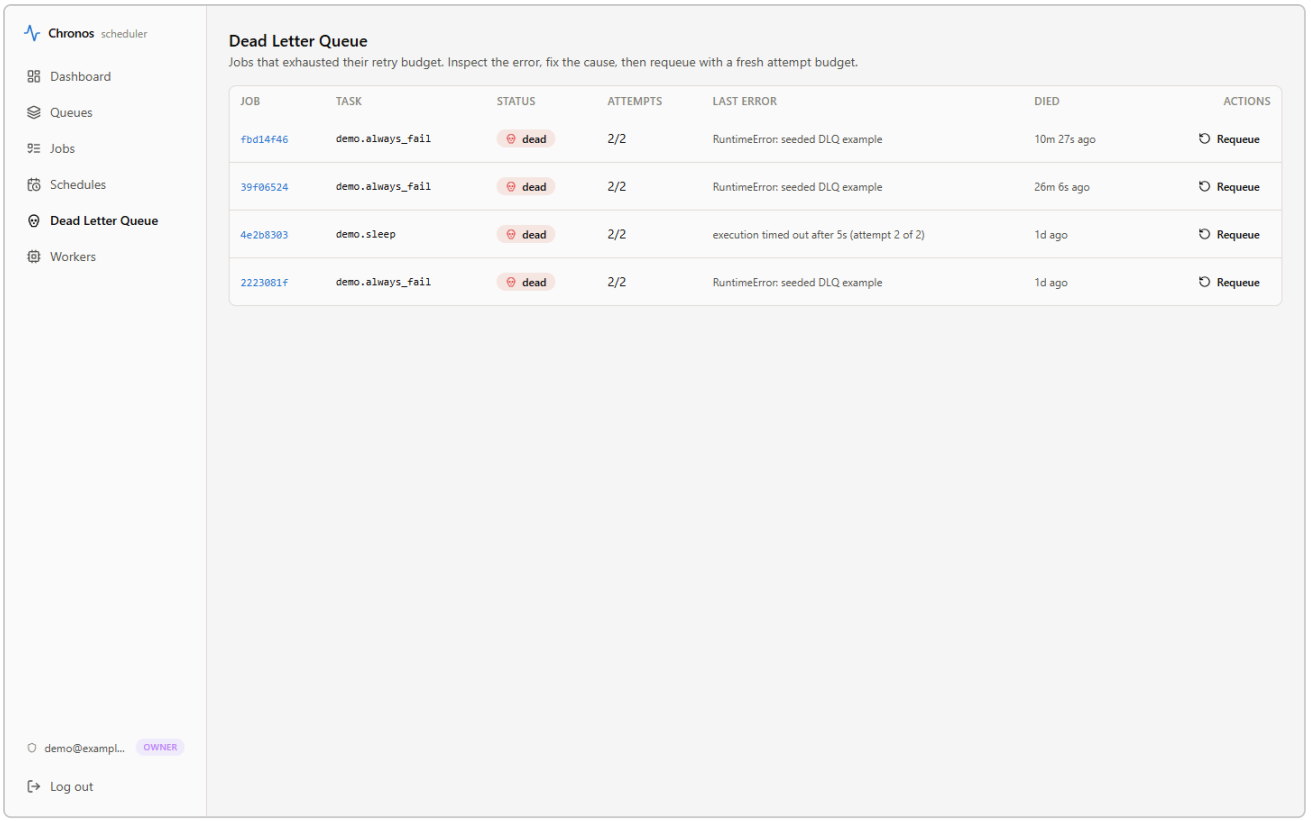
Job explorer — status/queue filters, pagination, priorities, attempts.



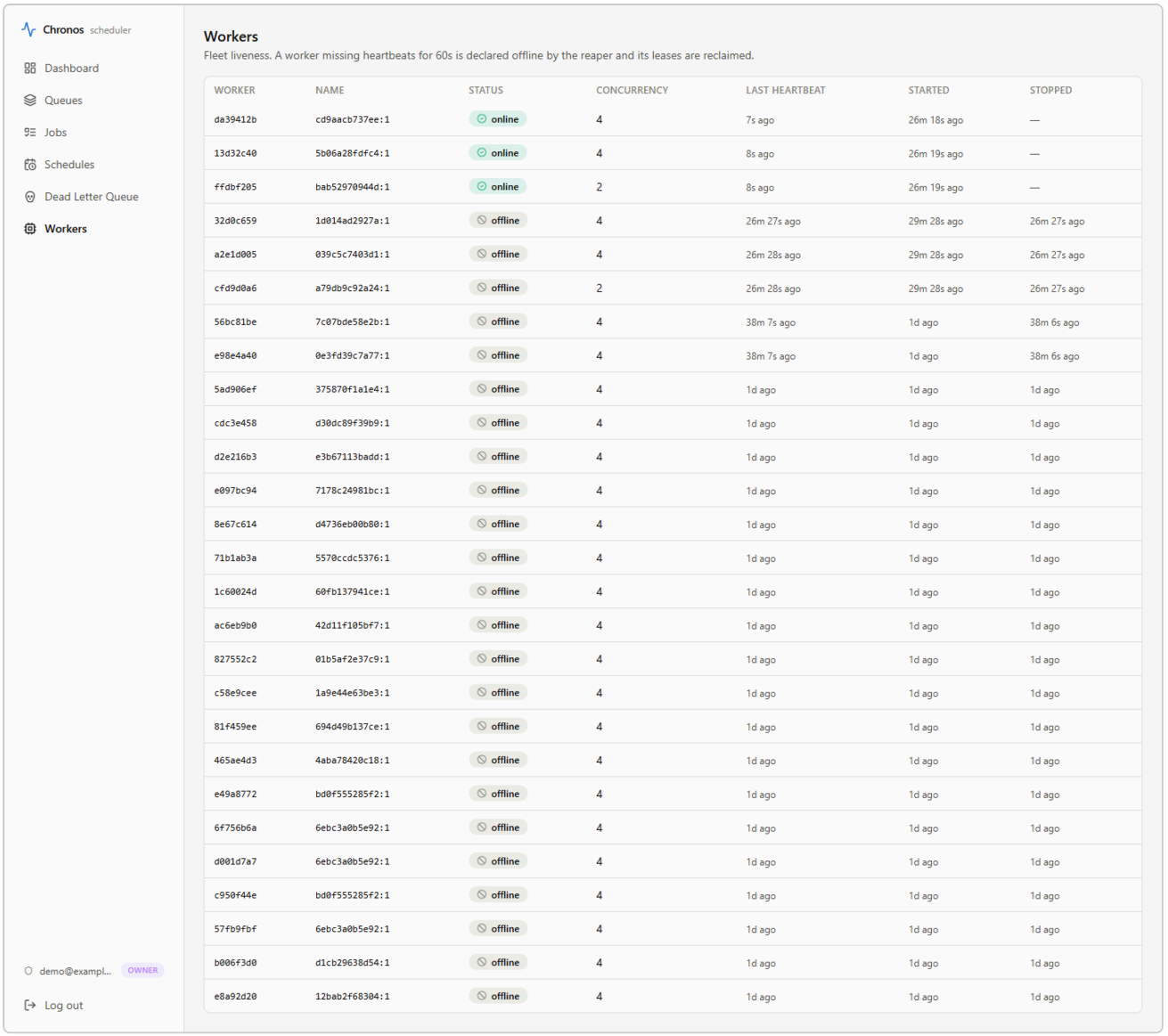
Job detail — a DLQ'd job: payload, full traceback, retry/backoff policy, per-attempt execution history with worker assignment and durations, one-click requeue.



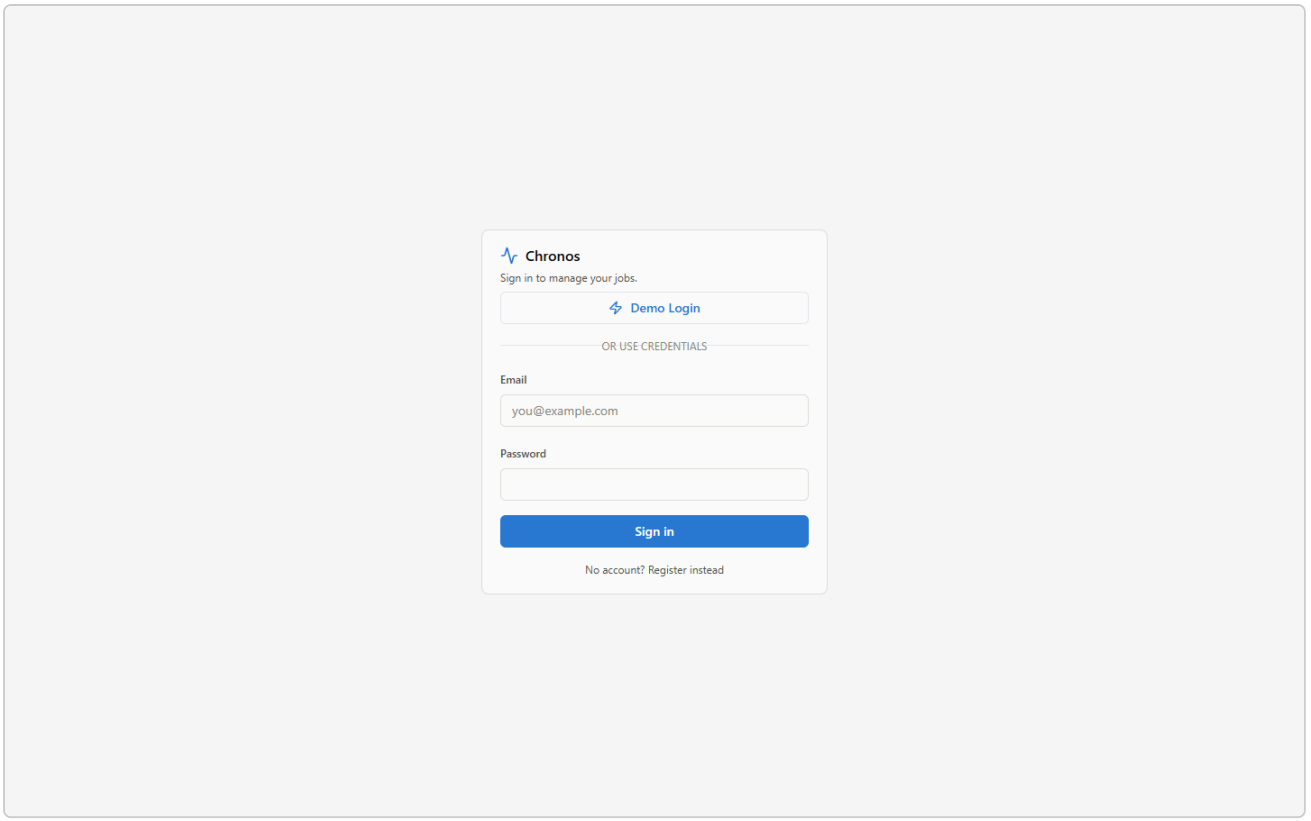
Recurring schedules — cron expression, next/last run cursors, pause/resume, create dialog with presets.



Dead Letter Queue — jobs that exhausted their retry budget; inspect and requeue with a fresh budget.



Worker fleet — liveness via heartbeats; a worker silent for 60s is declared offline and its leases reclaimed.



Login — JWT auth, registration, and one-click demo login.

API reference

Generated from the service's OpenAPI spec; the interactive version (request/response schemas, try-it-out) is served at `/docs`. All endpoints return structured errors, and list endpoints support `limit`/`offset` pagination plus field filters. 🔒 = bearer JWT required.

auth

Method	Path	Summary	
`POST`	`/api/v1/auth/login`	Login	
`GET`	`/api/v1/auth/me`	Me	🔒
`POST`	`/api/v1/auth/register`	Register	

dead letter queue

Method	Path	Summary	
`GET`	`/api/v1/dlq`	List dead jobs	🔒
`POST`	`/api/v1/dlq/{job_id}/requeue`	Requeue a dead job with a fresh attempt budget	🔒

jobs

Method	Path	Summary	
`POST`	`/api/v1/jobs`	Enqueue a job	🔒
`GET`	`/api/v1/jobs`	List my jobs	🔒
`POST`	`/api/v1/jobs/batch`	Enqueue a batch of jobs atomically	🔒
`GET`	`/api/v1/jobs/{job_id}`	Get a job	🔒
`GET`	`/api/v1/jobs/{job_id}/attempts`	Execution history of a job	🔒
`POST`	`/api/v1/jobs/{job_id}/cancel`	Cancel a pending job	🔒
`GET`	`/api/v1/jobs/{job_id}/dependencies`	List workflow dependencies of a job	🔒
`POST`	`/api/v1/jobs/{job_id}/requeue`	Requeue a dead or cancelled job	🔒

projects

Method	Path	Summary	
`GET`	`/api/v1/org`	My organization	🔒
`GET`	`/api/v1/projects`	List my organization's projects	🔒
`POST`	`/api/v1/projects`	Create a project	🔒
`GET`	`/api/v1/projects/{project_id}`	Get a project	🔒

queues

Method	Path	Summary	
`GET`	`/api/v1/queues`	List queues with per-queue job counts	🔒
`POST`	`/api/v1/queues`	Create a queue	🔒
`GET`	`/api/v1/queues/{queue_id}`	Get a queue	🔒
`PATCH`	`/api/v1/queues/{queue_id}`	Update queue configuration	🔒
`POST`	`/api/v1/queues/{queue_id}/pause`	Pause a queue	🔒
`POST`	`/api/v1/queues/{queue_id}/resume`	Resume a queue	🔒
`GET`	`/api/v1/queues/{queue_id}/stats`	Per-queue job statistics	🔒

schedules

Method	Path	Summary	
`POST`	`/api/v1/schedules`	Create a recurring (cron) schedule	🔒
`GET`	`/api/v1/schedules`	List my schedules	🔒
`GET`	`/api/v1/schedules/{schedule_id}`	Get a schedule	🔒
`PATCH`	`/api/v1/schedules/{schedule_id}`	Update a schedule	🔒
`DELETE`	`/api/v1/schedules/{schedule_id}`	Delete a schedule	🔒

stats

Method	Path	Summary	
`GET`	`/api/v1/stats/overview`	Cluster-wide scheduler metrics	🔒

users

Method	Path	Summary	
`GET`	`/api/v1/users`	List users in your organization	🔒
`PATCH`	`/api/v1/users/{user_id}/role`	Change a user's role (owner only)	🔒

workers

Method	Path	Summary	
`GET`	`/api/v1/workers`	List worker fleet	🔒

health

Method	Path	Summary	
`GET`	`/healthz`	Liveness probe	
`GET`	`/metrics`	Prometheus metrics	
`GET`	`/readyz`	Readiness probe (checks Postgres and Redis)	

Chronos — Engineering Design Document

Status	Implemented and verified; implementation frozen
Authors	Backend engineering
Reviewers	Internal design review
Repo	this repository (`backend/`, `frontend/`, `docker-compose.yml`)
Companion docs	`ARCHITECTURE.md` (rationale), `docs/PROJECT_REPORT.md` (summary), `docs/DIAGRAMS.md` (protocol diagrams)

---

1. Executive summary

Chronos is a distributed job scheduler: clients enqueue units of work — immediate, delayed, recurring (cron), or in atomic batches — over an authenticated HTTP API, and a horizontally scalable fleet of worker processes executes them with explicit reliability semantics — at-least-once execution, atomic claiming, lease-based failure detection, per-job retry policies (fixed, linear, or exponential backoff), a dead letter queue, idempotent enqueue, and graceful drain on shutdown. Work is organized as organization → project → queue; queues are first-class configuration objects carrying pause/resume, a fleet-wide concurrency cap, and default retry policy.

The central design commitment is that **PostgreSQL is the only source of truth**. Queue state, leases, retry policy, schedule cursors, and the execution audit trail live in eight tables; every state transition is a single guarded `UPDATE` whose `WHERE` clause restates the state the writer believes in, so a stale actor's write is rejected by the database rather than by convention. Redis exists only to reduce claim latency (a pub/sub wake channel) and dashboard load (a 3-second stats cache); its failure degrades latency by at most the 2-second poll interval and affects nothing else.

The system was verified empirically, not only by unit tests. Section 12 reproduces the results: exactly-once claim distribution under 8 concurrent claimers; a job that fails twice and succeeds on attempt 3; guaranteed DLQ delivery; duplicate enqueue suppression (HTTP 201 then 200, same row); recovery from `SIGKILL` of a worker mid-job (attempt recorded `lost`, retry succeeded on a peer ~35 s later); and `SIGTERM` drain that released an unfinished job back to the queue with its attempt refunded.

Scope was cut deliberately rather than thinly spread: no cancellation of running jobs, no multi-tenant fairness quotas, no per-schedule timezones (cron is UTC-only). Sections 13–14 quantify the scaling ceilings and order the improvements.

---

2. Problem statement

Web backends accumulate work that must not run inside a request: emails, webhooks, exports, billing operations, third-party API calls. The naive solutions fail in known ways:

- **In-process background tasks** die with the process and leave no record.
- **A Redis list + consumer** loses jobs on consumer crash (`BRPOP` removes the item before the work is done) unless a reliable-queue pattern is built by hand — at which point one is building a scheduler anyway, without transactions.
- **A message broker** (RabbitMQ/SQS) introduces the dual-write problem: the application row and the broker message cannot be committed atomically, so either an outbox pattern is built or the system tolerates phantom/lost jobs. Operational surface also grows by one stateful system.

The problem, precisely: **accept work transactionally, execute it on a fleet that machines can join and leave (or crash out of) at any time, never lose an accepted job, bound the damage of a poison job, make duplicate submissions safe, and keep every decision the system made inspectable after the fact.**

A second-order requirement shapes the design as much as the first-order ones: the failure semantics must be *explainable*. Every "what happens if X dies here" question should have a short answer derivable from a small number of invariants, because the operators of such a system are its first users.

---

3. Requirements

3.1 Functional

#	Requirement
F1	Authenticated users enqueue jobs: task name, JSON payload, queue, priority (−100..100), optional future `run_at`, retry policy
F2	Workers execute jobs concurrently; each ready job is handed to exactly one worker at a time
F3	Failed jobs retry per policy (max attempts; fixed, linear, or exponential backoff with cap and jitter)
F4	Jobs that exhaust their budget enter a dead letter queue; DLQ jobs can be inspected and requeued with a fresh budget
F5	Enqueue is idempotent under a client-supplied key (body field or `Idempotency-Key` header)
F6	Pending jobs can be cancelled; running jobs cannot (documented)
F7	Full execution history per job (which worker, when, outcome, error)
F8	Operations dashboard: cluster stats, throughput, queue/job/schedule/DLQ/worker views
F9	Tenancy: organization → project → queue; users manage projects and queues within their organization
F10	Queues are configurable: pause/resume, fleet-wide concurrency cap, default retry policy inherited by jobs that don't set their own
F11	Recurring (cron) schedules materialize ordinary jobs; missed ticks collapse into a single firing
F12	Batch enqueue: up to 100 jobs in one transaction, all-or-nothing, sharing a `batch_id`
F13	Workflow dependencies: a job with `depends_on` becomes claimable only after all its prerequisites SUCCEED
F14	Queue sharding: queues carry a `shard_key`; a worker started with `WORKER_SHARD` claims only its shard
F15	RBAC: owner/admin/member/viewer hierarchy enforced per endpoint
F16	AI failure summaries: DLQ errors get an operator-facing root-cause summary (Gemini; cached by error hash; absent key degrades to nothing)

3.2 Non-functional

#	Requirement	Realization
N1	No accepted job is ever lost, including on `SIGKILL`	lease expiry + reaper (§9, verified §12.3)
N2	No job is executed by two workers *concurrently believing they own it*	claim atomicity + fencing guards (§6)
N3	Worker fleet is elastic; joining/leaving requires no coordination beyond the database	workers self-register; reaper handles departure
N4	Clock skew between processes must not affect correctness	all time arithmetic in Postgres `now()` (§7.2)
N5	Loss of Redis must not affect correctness	wake channel is advisory; poll fallback (§4.3)
N6	Deploys must not burn retry budget or delay jobs by more than seconds	drain + release with attempt refund (§9.3)
N7	Every state transition attributable and auditable	append-only `job_attempts`, structured logs with correlation ids

3.3 Explicit non-goals

Exactly-once side effects (impossible across failure domains — §10 explains what is offered instead); cancellation of in-flight handlers; multi-region; per-tenant fairness/quotas; per-schedule timezones (cron is evaluated in UTC — DST-ambiguous local times are exactly the class of bug this system exists to avoid; clients localize for display); sub-second scheduling precision (claim granularity is the 2 s poll fallback, usually masked by the wake channel; schedule granularity is the 5 s materializer tick).

---

4. Architecture

4.1 Topology

Three roles share one codebase and one container image; behavior is selected by entrypoint. This eliminates model/schema drift between roles by construction.

- **API** (`uvicorn app.main:app`) — HTTP edge. Validates DTOs, enforces ownership, performs enqueue/read/cancel/requeue transactions. Executes no jobs.
- **Worker** (`python -m app.worker.main`, ×N) — runs three concurrent asyncio loops: *consume* (claim → execute → finalize), *heartbeat* (every 10 s), *reaper* (every 5 s, advisory-locked so one sweep runs fleet-wide).
- **PostgreSQL 16** — all durable state. **Redis 7** — wake channel + stats cache only.

Compose runs `postgres`, `redis`, `api`, `worker ×2`, `frontend`, with health-gated startup ordering (workers start only after the API is healthy, which is also what serializes schema migration — §4.4).

4.2 Backend layering

```
api/routers    HTTP concerns only: DTO validation, auth dependency,
               typed-error → status-code mapping. Zero business logic.
services       business rules and transaction boundaries. No HTTP types,
               no SQL strings. Raise NotFoundError/ConflictError/etc.
repositories   all SQL. The claim CTE, every guarded UPDATE, list/count
               queries. No policy decisions.
domain         pure functions. RetryPolicy → FailureDecision. No I/O.
models         SQLAlchemy 2 typed models; the schema's source of truth.
schemas        Pydantic DTOs. ORM objects never cross the HTTP boundary.
```

Two facts keep this honest rather than ceremonial. First, the worker process imports `services/` and `repositories/` but not FastAPI — the service layer demonstrably has no HTTP dependency. Second, the retry decision used by both the worker's failure path and the reaper is one pure function (`app/domain/retry.py`); the two failure paths cannot diverge because there is only one implementation to call.

Session discipline differs by role, deliberately: the API uses one dependency-injected session per request; the worker's `ExecutionService` owns a session factory and opens a short transaction per operation, so a 90-second job holds a database connection for milliseconds, not minutes.

4.3 Redis: the degradation contract

Every Redis interaction is wrapped so that failure is converted, never propagated: `publish_wake` catches, logs, drops; `wait_for_wake` converts any Redis error into `sleep(poll_interval)`; the stats cache becomes a pass-through. The wake is published strictly **after** the enqueue commit — publishing before commit would race workers against a row they cannot see. Consequence: Redis outage costs ≤ 2 s of claim latency and some dashboard query load, and cannot cost correctness. This was a requirement (N5), not an accident, and the readiness probe reflects it: `/readyz` gates on Postgres only and reports Redis as informational.

4.4 Migration execution

`alembic upgrade head` runs in the API entrypoint; workers do not migrate. Compose's health-gated ordering means exactly one process applies DDL before any worker connects. Known limit: this assumes one API replica; scaling the API horizontally requires extracting a one-shot migrate job (§14).

---

5. Database design

5.1 Schema

Nine tables (ER diagram: DIAGRAMS.md §11). The design principle: `jobs` is a narrow, update-hot state-machine record; everything historical is append-only elsewhere, and everything configurational lives on the entity that owns it (queue rows), snapshotted onto jobs at enqueue.

`organizations` / `projects` — the tenancy chain. Every user gets a personal org and a `default` project at registration (one transaction), so "which project?" always has an answer and enqueue never needs a setup

dance. Access control is a single org-id comparison; `projects` are unique by `(org_id, name)`.

`queues` — first-class configuration objects, unique by `(project_id, name)`, auto-created on first use. Operator controls (`paused`, `max_concurrency`, `shard_key`) are read *inside the claiming transaction*, so a pause takes effect at the next claim with no worker coordination; capped queues are admitted under `FOR UPDATE` of the queue row, making the running-count check and the admission atomic (§6). Default retry policy lives here and is copied to jobs at enqueue — editing a queue never mutates in-flight work.

`schedules` — recurring (cron) work: a job template plus a cursor (`next_run_at`, partial-indexed on unpaused rows). The materializer (an advisory-locked sweep hosted by every worker, like the reaper) turns due schedules into ordinary `jobs` rows; firing is exactly-once because the insert and the cursor advance commit atomically, backstopped by the deterministic idempotency key `schedule:<id>:<fire-time>`. Missed ticks collapse into a single firing.

`jobs` — identity (`owner_id`, `queue_id` + denormalized `queue` name, `task_name`, `payload JSONB`, optional `batch_id` / `workflow_id`), scheduling (`status`, `priority`, `run_at`), execution policy denormalized as six columns (`backoff_strategy` — fixed/linear/exponential — `max_attempts`, `timeout_seconds`, `backoff_base_seconds`, `backoff_factor`, `backoff_max_seconds` — copied at enqueue so a default change never mutates in-flight jobs), lease (`locked_by → workers`, `lease_expires_at`), outcome (`result JSONB`, `last_error`, optional `ai_summary`), timestamps. `attempt_count` lives here because the claim increments it atomically. The queue *name* is denormalized because the worker fleet subscribes by name across projects; the claim scan filters on the name and joins the row for its controls.

`job_dependencies` — workflow edges (`job_id` depends on `depends_on_job_id`, unique per pair). The claim scan excludes jobs with unmet dependencies, so a dependent job becomes claimable the instant its last prerequisite succeeds — no orchestrator process.

`job_attempts` — one row per claim, opened in the claim transaction, closed exactly once as `succeeded | failed | lost | aborted` with error text and timing. This is the audit trail (F7) and the source for throughput metrics. Closing updates are guarded by `status = 'running'`, so a verdict, once written, is immutable — the reaper and a slow worker cannot both write one.

`workers` — fleet registry: `name`, `status (online|draining|offline)`, `concurrency`, `last_heartbeat_at`, `started_at`, `stopped_at`. Observational only: nothing in the correctness path reads it (§7.1).

`users` — email (unique, normalized), bcrypt hash, `org_id`, and an RBAC `role` (owner > admin > member > viewer, checked by rank at the API edge: job mutations require member+, queue configuration admin+, role management owner).

5.2 State machine

```
enqueue → PENDING → (claim) → RUNNING → SUCCEEDED
             ↑                   │  └↳ PENDING   (fail, budget left; run_at = now()+backoff)
             │                   └↳ DEAD         (budget exhausted)      ← the DLQ
             ├── CANCELLED ←— (cancel, PENDING only)
             └── (requeue from DEAD/CANCELLED, attempt budget reset)
```

A retry is **not a status** — it is `PENDING` with a future `run_at` and `attempt_count > 0`. The claim predicate `run_at <= now()` therefore *is* the retry scheduler and the future-scheduling feature simultaneously; there is no timer wheel or delayed-delivery mechanism to build, monitor, or crash. Native PG enums make illegal statuses unrepresentable; CHECK constraints bound every policy field (`max_attempts 1..20`, `priority -100..100`, etc.).

5.3 Indexing

Indexes were designed per query on a write-hot table, where each extra index taxes claim/heartbeat/finalize with WAL and page churn:

Index	Type	Serves
`ix_jobs_claim (priority DESC, run_at) WHERE status='pending'`	partial	claim CTE — matches its predicate and sort exactly; **size tracks backlog, not history**, so claim cost is independent of total table size
`ix_jobs_lease (lease_expires_at) WHERE status='running'`	partial	reaper's expiry scan
`uq_jobs_owner_idempotency (owner_id, idempotency_key) WHERE idempotency_key IS NOT NULL`	unique partial	idempotent enqueue — the index is the correctness mechanism, not an optimization (§10); also the exactly-once backstop for schedule firings
`ix_jobs_owner_created (owner_id, created_at DESC)`	btree	owner-scoped listing
`ix_jobs_queue_running (queue_id) WHERE status='running'`	partial	concurrency-cap admission count — proportional to in-flight work, not history
`ix_jobs_batch (batch_id) WHERE batch_id IS NOT NULL`	partial	batch tracking
`ix_jobs_workflow (workflow_id) WHERE workflow_id IS NOT NULL`	partial	workflow views
`ix_schedules_due (next_run_at) WHERE NOT paused`	partial	the materializer's scan
`ix_job_deps_depends_on (depends_on_job_id)`	btree	dependency-gate subquery in the claim scan
`ix_job_attempts_job_number`, `ix_job_attempts_finished`	btree	attempt history; throughput-by-minute

Deliberately absent: indexes on `task_name`, bare `queue`, `payload` (no query needs them), and — a known, accepted gap — `(locked_by) WHERE status='running'` for the heartbeat, justified only at fleet sizes we have not reached.

The attempt-number index is deliberately **non-unique**: a DLQ requeue resets `attempt_count`, so numbering restarts; old rows are terminal and all attempt updates are guarded by `status='running'`, keeping writes unambiguous. The alternative (an epoch column) is listed in §14.

5.4 Conventions

All constraints and indexes are named through a deterministic naming convention, so production DDL and migration review are predictable. The initial migration is hand-written, not autogenerated, and reviewable as DDL.

---

6. Concurrency model

6.1 The claim

```
WITH candidates AS (
    SELECT id FROM jobs
    WHERE status = 'pending' AND run_at <= now() AND queue = ANY(:queues)
    ORDER BY priority DESC, run_at, created_at
    LIMIT :free_slots
    FOR UPDATE SKIP LOCKED
)
UPDATE jobs
SET status='running', locked_by=:worker_id,
    attempt_count = attempt_count + 1,
    started_at = now(), lease_expires_at = now() + :lease
FROM candidates WHERE jobs.id = candidates.id
RETURNING jobs.*;
```

Three properties, each load-bearing:

1. **`SKIP LOCKED`** — concurrent claimers pass over rows another transaction holds instead of blocking or double-claiming; N claimers partition the ready set. The accepted cost is best-effort ordering under contention (a locked higher-priority row is invisible for the duration of the other claimer's transaction).
2. **The CTE form** — Postgres may re-evaluate a subquery in `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED)`; a CTE with a locking clause is evaluated once, pinning "rows locked" and "rows updated" to the same set.
3. **Same-transaction audit** — the `job_attempts` row is inserted before commit, so a claim without an audit record cannot exist under any crash timing.

6.2 Fencing without tokens

Every transition out of `RUNNING` is:

```
UPDATE jobs SET ... WHERE id = :id AND status = 'running' AND locked_by = :me
```

The `(status, locked_by)` pair functions as a fencing token whose comparison executes inside the same ACID domain as the write. Classic fencing tokens exist because the protected resource (a file server, an S3 bucket) cannot evaluate lock ownership itself; here the protected resource *is* the row, so the guard predicate does the job with no extra machinery. **There are no blind writes in the system.** This is the

single property that makes every interleaving analyzable: whichever of two racing actors commits second finds its precondition false and becomes a rowcount-0 no-op.

Worked race (verified live, §12.3): reaper locks an expired job with `FOR UPDATE`; the zombie worker's completion UPDATE blocks on the row lock; reaper commits `status='pending', locked_by=NULL`; the worker's predicate re-evaluates under `READ COMMITTED`, matches nothing; the worker logs `lease_lost` and discards its result. In the reverse order, the job is no longer `running` so the reaper's expiry scan never selects it. If the finalizer holds the row lock uncommitted while the reaper scans, the reaper's own `SKIP LOCKED` skips the row rather than contending.

6.3 The reaper singleton

Any worker may sweep; `pg_try_advisory_xact_lock(key)` admits one per tick. The transaction-scoped variant matters: the lock dies with the transaction, so a reaper that crashes mid-sweep releases the lock *by crashing* — there is no lock-leak failure mode and no session-pooling hazard. Failover is implicit (any survivor wins the next 5 s tick). Strictly, the sweep's `FOR UPDATE SKIP LOCKED` would keep concurrent sweeps safe anyway; the advisory lock buys efficiency (no duplicate scans) and clean semantics ("one sweep at a time" is a stated invariant, not an emergent one).

6.4 Worker execution model and its stated limit

One asyncio event loop per worker; up to `WORKER_CONCURRENCY` (default 4) handler coroutines plus the two maintenance loops. Correct and efficient for I/O-bound handlers — the design target.

Every attempt runs under `asyncio.wait_for(handler, job.timeout_seconds)` (default 300 s, per-job column, CHECK-bounded 1..86400). A timeout is a normal failure: `complete_failure` records the reason ("execution timed out after Ns (attempt i of m)") on the attempt row and the job, and the shared retry/DLQ decision applies unchanged. This bounds the hung-handler case — without it, the heartbeat would renew a stuck job's lease forever. Shutdown semantics are preserved: outer-task cancellation propagates through `wait_for` as `CancelledError`, so the drain path still releases with an attempt refund (pinned by test, §12.1a). Caveat: `wait_for` cancellation is cooperative — a handler that swallows `CancelledError` can still hang.

The remaining limit, stated plainly: **heartbeat liveness depends on the event loop getting scheduled.** A CPU-bound handler starves the loop; heartbeats stop; after 30 s the lease expires under a still-running job; the reaper reschedules it; side effects can then execute twice (state cannot diverge — the zombie's finalize is fenced off). The fix is a process-pool execution lane, ranked in §14.

---

7. Reliability model

7.1 Liveness: leases + heartbeats

A claim writes `lease_expires_at = now() + 30 s`. Every 10 s a worker, in **one transaction**, updates its own `last_heartbeat_at` and extends `lease_expires_at` for every job it holds — worker liveness and job leases move together or not at all, so there is no observable state where the fleet view and the lease state disagree.

The 3:1 lease-to-heartbeat ratio is the tuning invariant (`LEASE_SECONDS ≥ 3 × HEARTBEAT_SECONDS`): one missed heartbeat — a GC pause, one dropped packet, one failed transaction — must never trigger a reclaim, because a reclaim risks duplicate side effects and burns an attempt. Three consecutive misses is treated as death.

Correctness never reads the `workers` table; the per-job lease is the ground truth and the worker row is derived, observational state (dashboard, offline marking). One authority per fact.

7.2 Time

Every temporal comparison — `run_at <= now()`, lease expiry, backoff scheduling — executes in Postgres. Worker and API wall clocks appear nowhere in correctness logic. Consequence: container clock skew cannot produce premature reclaims, immortal leases, or early/late scheduling; the worst it can do is mislabel log timestamps.

7.3 Delivery semantics

At-least-once, with the irreducible window stated: a worker that performs a side effect and dies before `finish_success` commits leaves a job that will re-run. Acknowledge-before-execute (at-most-once) silently loses work instead — worse for every workload this system targets. What Chronos guarantees exactly-once is *acceptance* (idempotent enqueue, §10) and *state transitions* (fencing, §6.2). Handlers receive `job_id` in their context as the deduplication scope for downstream effects.

---

8. Retry strategy

Policy lives on the job row (denormalized at enqueue): `max_attempts` (default 3), `backoff_base_seconds` (5), `backoff_factor` (2.0), `backoff_max_seconds` (300). The decision function is pure:

```
delay(n) = min(base × factor^(n−1), cap) + min(...) × 0.2 × U(0,1)
retry    = (n < max_attempts)
```

- **Exponential with cap**: uncapped, attempt 10 at factor 2 waits ~42 minutes; capping at 300 s keeps time-to-DLQ-verdict bounded.
- **Jitter (0–20%)**: failures are correlated — a downstream outage fails hundreds of jobs in the same second, and deterministic backoff would re-synchronize them into retry waves hammering a recovering dependency. Jitter decorrelates the waves.
- **One implementation, two callers**: the worker's failure path and the reaper's lease-expiry path call the same `decide_failure`. Retry semantics cannot fork between "handler raised" and "worker died."

Two attempt-accounting rules with opposite signs, both deliberate:

1. **Lease expiry consumes an attempt.** A poison job that crashes or wedges its workers must converge to the DLQ; if reclaims were free, it would cycle through the fleet forever. Cost accepted: a job that was innocent of its worker's death (OOM caused by a different job) also burns one.
2. **Graceful-shutdown release refunds the attempt** and reschedules at `run_at = now()`. Deploys are operator actions carrying zero evidence against the job; counting them would push long-running

jobs toward the DLQ as a function of deploy frequency. The audit trail distinguishes the cases: `lost` (expiry) vs `aborted` (release).

The DLQ is `status = 'dead'`, not a separate table: a dead job keeps its payload, error, and complete attempt history, and requeue (`DEAD → PENDING`, budget reset) is one guarded transition rather than a cross-table move.

---

9. Failure recovery

9.1 The recovery funnel

There is exactly **one** recovery mechanism — lease expiry → reaper — and every crash window funnels into it:

Crash point	Recovery
After claim commit, before execution	lease expires; attempt closed `lost`; retried. The attempt row exists (claim-transaction atomicity)
Mid-execution	identical — the DB cannot and need not distinguish
After side effect, before finalize commit	identical; side effect may duplicate (§7.3)
During finalize transaction	rolls back; identical to the previous row
During graceful drain	released jobs are already `pending`; unreleased ones fall through to lease expiry — the graceful path degrades *into* the crash path, never below it

9.2 The reaper sweep

Every 5 s, under the advisory lock: select up to 100 jobs where `status='running' AND lease_expires_at < now()` with `FOR UPDATE SKIP LOCKED`; for each, close the attempt as `lost` (guarded), run `decide_failure`, and either reschedule with backoff or move to `DEAD`; finally mark workers with `last_heartbeat_at` older than 60 s as `offline`. Mass-death arithmetic: 100 per 5 s ≈ 20 reclaims/s; a 5,000-lease correlated failure fully recovers in ~4 minutes (improvement: loop the sweep within a time budget, §14).

9.3 Graceful shutdown

On `SIGTERM`: (1) stop claiming and mark `draining` — visible in the fleet view mid-deploy; (2) wait up to 20 s for in-flight jobs, **with heartbeats still running** so their leases stay alive through the grace window (stopping heartbeats at drain start would let a 25-second-old claim expire mid-drain and be double-executed by a peer — a shutdown-manufactured race); (3) cancel the remainder and release each: `status='pending'`, `run_at=now()`, `attempt_count −1`, attempt row `aborted`; (4) mark `offline`, close connections, exit. Compose's `stop_grace_period` (30 s) deliberately exceeds the drain window (20 s) so Docker never `SIGKILL`s a draining worker mid-release; if it ever does, §9.1's funnel catches the stragglers.

Verified sequence in §12.4.

---

10. Idempotency

Enqueue. The client supplies a key (DTO field or `Idempotency-Key` header). The authority is the unique partial index `(owner_id, idempotency_key) WHERE idempotency_key IS NOT NULL` — the service's pre-check is a fast path only and is not trusted. Two concurrent submissions both INSERT; the index admits one; the loser catches `IntegrityError`, rolls back, re-reads by key, and returns the winner's row. The API distinguishes outcomes: 201 for created, 200 for deduplicated — verified live (§12.2) with identical job ids. Owner-scoping makes keys a private namespace per user (no cross-tenant collision or probing).

Execution. At-least-once delivery makes handler-level idempotency the caller's contract; the system supports it by passing `job_id` and `attempt` in the handler context (the natural downstream deduplication key) and by never re-running a job that reached a terminal state (fencing).

Known gaps, acknowledged: keys never expire, and re-use of a key with a *different payload* silently returns the original job rather than failing with 409 (Stripe semantics: store a payload hash, reject mismatches). Both are listed in §14.

---

11. Tradeoffs

Each row records the decision, the road not taken, and the accepted cost.

Decision	Rejected alternative	Accepted cost
Postgres as queue	Redis/RabbitMQ/SQS/Kafka	claim ceiling ~10^3/s; poll latency (masked by wake channel). Bought: transactional enqueue, no dual-write, SQL-queryable everything
At-least-once + idempotency	at-most-once; "exactly-once" claims	duplicate side effects in the crash-after-effect window. Bought: no silent work loss; honest contract
Leases + heartbeats, DB clock only	worker-clock TTLs; external lock service	~35 s worst-case failure detection. Bought: skew immunity; no new infrastructure
Guarded UPDATEs as fencing	monotonic fencing tokens; version columns	none within the DB; external side effects remain unfenced (inherent). Bought: zero extra machinery
Retry = `PENDING` + future `run_at`	delayed queues; timer service	retry latency quantized by poll interval; "retrying" is inferred, not a status. Bought: scheduler = one WHERE clause
Expiry burns an attempt; release refunds	SQS-style separate delivery counter	innocent jobs can burn budget in correlated crashes. Bought: poison-pill convergence with minimal accounting
Reaper in workers + advisory lock	dedicated scheduler service; leader election	reaper capacity coupled to fleet deployment. Bought: one less deployable; implicit failover; crash-releases-lock semantics
DLQ as a status	separate DLQ table/queue	terminal rows share the hot table until archival exists. Bought: identity/audit preservation; requeue is one transition
One image, three roles	separate API/worker services	coupled deploys; migrations must be cross-role compatible. Bought: zero code drift
No cancel of running jobs	cooperative cancel flags; task killing	operator must drain a worker to free a stuck slot (until per-job timeouts land). Bought: no half-executed side effects from a kill path

---

12. Testing results

All results below were produced against this repository — the unit suite, the concurrency test against real Postgres, and fault injection against the running compose stack. Full suite: **9/9 passing**.

12.1 Unit: retry policy (pure domain, no infrastructure)

8 tests pin the behavior both failure paths share: exponential growth (5 → 10 → 20 s at factor 2), the 300 s cap, the jitter bound (≤ 20%, verified at the extremes with a stubbed RNG), DLQ decision at `n = max_attempts`, rejection of invalid attempt numbers, and the `max_attempts = 1` never-retry edge.

12.1a Unit: execution timeouts and outcome routing

4 tests drive the real `WorkerRunner._execute` against a stub execution service (no DB/Redis): a 30 s handler under a 1 s budget routes to `complete_failure` with "timed out after 1s (attempt 1 of 3)" in the recorded reason; a fast handler under budget routes to `complete_success`; a raising handler still routes to failure; and cancelling the outer task routes to `release_job` with nothing recorded as failed — the proof that `wait_for` did not alter graceful-shutdown semantics.

12.2 Integration and API-level verification

- **Claim atomicity:** 8 concurrent claimers, 50 jobs, batches of 5, real Postgres. Assertions: no job id appears twice (no double-claim) and the union equals the created set (no starvation). The test isolates on a unique queue so it runs safely against a live cluster, and deletes its rows afterward. Passing.
- **Idempotency:** two `POST /jobs` with `Idempotency-Key: verify-idem-1` → `HTTP 201` then `HTTP 200`, same id (`ace6b9b6-…`), one row.
- **Retry pipeline (seeded demo jobs, observed end state):** `demo.fail_until{succeed_on_attempt: 3}` → `succeeded`, `attempt_count 3`; `demo.flaky{0.5}` → `succeeded` on attempt 2; `demo.always_fail` (budget 2) → `dead`, in the DLQ, requeue-able.

12.3 Fault injection: worker crash (`SIGKILL`)

A `demo.sleep{seconds: 90}` job was enqueued; the worker container holding its lease was identified by worker id and killed with `docker kill` (no signal handling). Observed in the database ~35 s later (lease 30 s + sweep):

```
attempt 1: lost   — "lease expired (worker … presumed dead)"
job:       pending, attempt_count 1, last_error set
```

then, after workers were restarted:

```
attempt 2: succeeded
job:       succeeded, result {"slept_seconds": 90.0}
```

Separately, 8 abandoned claimers (from the concurrency test) holding 50 leases with no heartbeats were fully reclaimed by the reaper, and 18 stale worker rows were flipped to `offline` — the mass-death path exercised incidentally and behaving as designed.

12.4 Fault injection: graceful shutdown (`SIGTERM`)

A `demo.sleep{seconds: 300}` job (cannot finish within grace) was running when `docker compose stop worker` fired. Observed:

- structured log sequence: `worker.draining {inflight: 1}` → 20 s grace → `execution.job_released` → `job.released_on_shutdown` → `worker.stopped`;
- database: job `pending` with `attempt_count 0` (the claim's increment refunded), attempt row `aborted` with reason "worker shut down before the job finished"; both workers `offline` with `stopped_at` set;
- the idle worker drained in < 1 s (no inflight → immediate stop).

12.5 Benchmarks (measured 2026-07-04; full method in BENCHMARKS.md)

309 / 614 / 1,014 / 1,505 jobs/s at 1 / 2 / 4 / 8 workers over no-op handlers — linear to 2 workers, database contention bending the curve beyond, converging on the §13 prediction. Idle-fleet dispatch latency (enqueue → claimed) 35 ms p50 / 54 ms p95; execute+finalize 6–10 ms p50. The first run measured ~2 jobs/s and exposed a real defect — a saturated worker waited out the full poll interval instead of re-claiming when a job finished; fixed (completion now signals the consume loop) and re-measured at 150× the broken figure.

12.6 Not tested, stated

Handler business logic (demo tasks are scaffolding); long soak runs (benchmarks are single-host bursts); API contract tests beyond the verified flows above.

---

13. Scaling limits

Quantified ceilings, and what specifically breaks:

1. **Claim throughput — now measured: ~1,500 jobs/s at 8 workers on a single shared host** (§12.5), with contention already bending the curve (4.87× at 8×). Not the SELECT — the partial index keeps candidate scans O(backlog). The binding constraint is write amplification: ≥ 3 UPDATEs per job lifetime produce dead tuples and index churn faster than autovacuum defaults reclaim them; p99 claim latency is the leading indicator, dead-tuple ratio the confirming one. Head-of-queue lock contention among claimers adds lock-manager traffic.
2. **Unbounded table growth.** Terminal `jobs` and `job_attempts` rows accumulate forever. First casualty is the stats `GROUP BY` over `jobs` (every 3 s under dashboard load once the cache misses); second is vacuum and backup duration.
3. **Failure detection ≈ 35 s** (lease 30 + sweep 5). Tunable downward only against the 3:1 heartbeat invariant — sub-10 s detection requires accepting more false reclaims or per-job leases.
4. **Mass-death recovery ≈ 20 leases/s** (sweep cap 100 / 5 s).
5. **Single Postgres = availability ceiling.** By design: it is the source of truth. HA is a Postgres problem (replica + failover), not a scheduler problem — but today it is unaddressed.
6. **Stats fan-out.** Cluster-wide counts are recomputed per cache miss; with many dashboard viewers and Redis down, this becomes the top query.

---

14. Future improvements

Ordered by value per unit effort; each preserves the core property (transactional, guarded, auditable state transitions):

1. ~~Per-job execution timeout~~ — **implemented after review** (`timeout_seconds` column, migration 0002, `asyncio.wait_for`; timeout = failed attempt; shutdown semantics pinned by test §12.1a).
2. ~~Operational hardening~~ — **largely implemented**: `restart: unless-stopped` on all services; boot refusal when `JWT_SECRET` is the default outside development; 64KB payload cap at the DTO

edge; pinned `requirements.lock` + `package-lock.json` with `npm ci`; worker healthcheck keyed to heartbeat-file recency. Remaining: container resource limits; the patched Next.js release.
3. ~~Metrics endpoint~~ — **implemented**: Prometheus `/metrics` with jobs by status, ready/scheduled depth, **oldest-ready-job age**, claim-wait avg/max (5m), lease reclaims (1m), heartbeat lag, workers online, DLQ size. Remaining: alert rules and a scrape config (deployment concern).
4. **Retention:** nightly archival of terminal rows past N days; replace the global `GROUP BY` with a transition-maintained counter table.
5. **CPU lane:** process-pool execution for CPU-bound handlers, removing the event-loop-starvation duplicate-execution window (§6.4).
6. **Idempotency refinements:** payload-hash mismatch → 409; optional key TTL. **Attempt epochs** to disambiguate requeue history.
7. **Fairness:** ~~priority aging~~ — **implemented** (+1 effective priority per aged interval, capped; starvation bounded at interval × boost). Remaining: per-owner claim quotas. Per-IP login throttling also landed (Redis fixed window, fails open).
8. **Scale-out sequence:** per-queue claim sharding → status/time partitioning → and only then a broker with an outbox table, which reintroduces the dual-write problem deliberately and pays for it with a relay component. Each earlier step is roughly an order of magnitude cheaper than the next.
9. ~~**Recurring schedules**~~ — **implemented**: a `schedules` table materializes `jobs` rows via an advisory-locked sweep (one winner per tick, exactly-once firing via the idempotency index, missed ticks collapse). Remaining refinement: per-schedule timezones, deliberately excluded (UTC-only, §3.3).

---

15. Conclusion

Chronos demonstrates that a small system can have precise failure semantics. The design reduces to four invariants, each enforced by one mechanism:

1. **A ready job is handed to exactly one worker at a time** — the `SKIP LOCKED` claim CTE.
2. **Only the current lease holder can finalize a job** — guarded UPDATEs; no blind writes exist.
3. **Every accepted job reaches a terminal state or the queue again, with an audit row explaining each attempt** — the lease-expiry funnel and the claim-transaction attempt insert.
4. **Time is judged by one clock** — Postgres `now()`.

Everything else — retries, the DLQ, graceful drain, idempotent enqueue — is composition on top of those four. The claimed behaviors were exercised against the running system, including `SIGKILL` and `SIGTERM` fault injection, and behaved as specified (§12). The known weaknesses are stated with the same precision as the guarantees (§6.4, §13), and the improvement path (§14) is ordered so the cheapest fixes close the sharpest edges first.

The review question this document should equip you to ask of any alternative design is the one Chronos was built around: *for each place a process can die, show me the row, the guard, and the sweep that make it somebody else's job within a bounded time.*

Benchmarks

Measured on the docker-compose stack (single host, Postgres 16, Redis 7), 2026-07-04. Method: `app/scripts/bench.py` bulk-inserts N `demo.echo` jobs (no handler work — the numbers measure the *scheduler*: claim, dispatch, finalize round-trips), waits for the fleet to drain them, and computes stats from the database's own timestamps. Each run cleans up after itself. Reproduce with:

```
docker compose up -d --scale worker=N worker
docker compose exec api python -m app.scripts.bench --jobs 2000
```

Throughput scaling (2,000 jobs; 4,000 at 8 workers)

Workers (×4 concurrency)	Throughput	Scaling vs 1 worker	exec+finalize p95
1	309 jobs/s	1.00×	7 ms
2	614 jobs/s	1.99×	7 ms
4	1,014 jobs/s	3.28×	10 ms
8	1,505 jobs/s	4.87×	13 ms

Throughput scales linearly to 2 workers and sub-linearly beyond, with database contention becoming dominant around ~1,000–1,500 claims/sec on this host — consistent with the design doc's predicted "low thousands/sec" ceiling for a single-Postgres queue. The rising finalize p95 (7 → 13 ms) is the contention signature: more claimers means more lock traffic on the queue head and more round-trip queuing in Postgres. Past this point the design calls for per-queue sharding, then partitioning, then a broker with an outbox (see ARCHITECTURE.md, scaling path).

Latency

Measurement	Value	Condition
Dispatch latency (enqueue → claimed), p50 / p95	**35 ms / 54 ms**	idle fleet, 50 jobs into 64 free slots — no queueing; this is the Redis-wake fast path
Claim wait under saturation, p50	1.1–3.3 s	2,000-job burst against 8–64 slots; queue depth, not dispatch cost (Little's law)
Execution + finalize, p50	6–10 ms	echo handler; two guarded UPDATEs + attempt close
Retry latency (failure → next attempt running)	backoff + ~1 s	measured on the timeout demo: attempt 1 → attempt 2 in 10.0 s with a 5 s timeout + ~3.6 s jittered backoff
Crash recovery (SIGKILL → attempt marked `lost`)	~35 s	lease 30 s + reaper sweep ≤ 5 s; measured in live fault injection
Graceful shutdown, idle worker	< 1 s	SIGTERM → offline
Graceful shutdown, busy worker	≤ grace (20 s) + release ~10 ms	job back to `pending` with attempt refunded

What benchmarking found (and fixed)

The very first run measured **~2 jobs/s** on one worker — 150× below expectation. Root cause: when a claim filled every concurrency slot, the consume loop blocked on the Redis wake channel for the full 2 s poll interval, and nothing woke it when an in-flight job finished. Fast jobs therefore dispatched at `concurrency / poll_interval` (4 per 2 s) regardless of capacity. The fix (`runner.py`): job-completion callbacks set a `slot_freed` event, and the idle branch waits on *wake OR freed slot OR poll timeout*, whichever fires first. Same run, same hardware, after the fix: **309 jobs/s** — a 150× improvement that unit tests and fault injection had no chance of catching. Throughput claims that were never measured are fiction; this one was fiction until 2026-07-04.

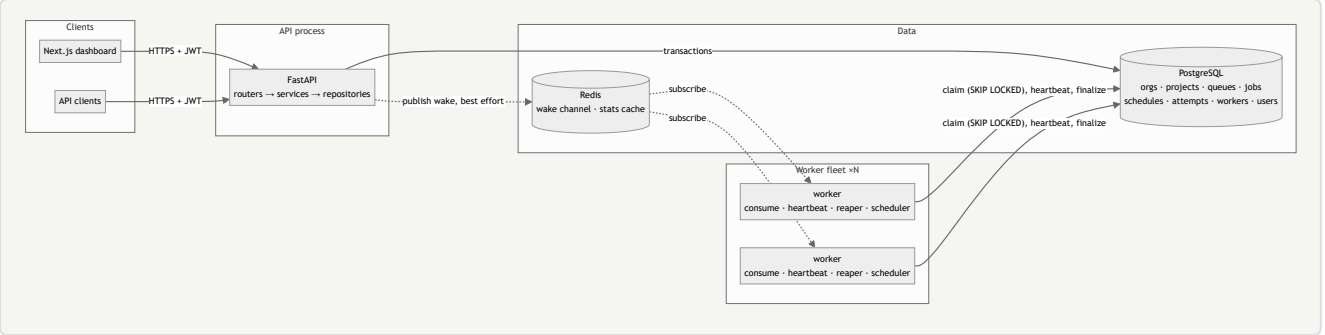
Caveats

Single host: workers, Postgres, and Redis share CPU, so the 8-worker row understates what a real fleet against a dedicated Postgres would do, and all numbers include compose networking. Percentiles come from `percentile_cont` over per-job timestamps written by Postgres itself (`now()` at claim and finalize), so they inherit ~ms timestamp resolution but no client clock skew. Echo jobs are the scheduler's worst case per unit of useful work — real workloads with seconds-long handlers hit the concurrency ceiling (`workers × WORKER_CONCURRENCY`) long before the claim ceiling.

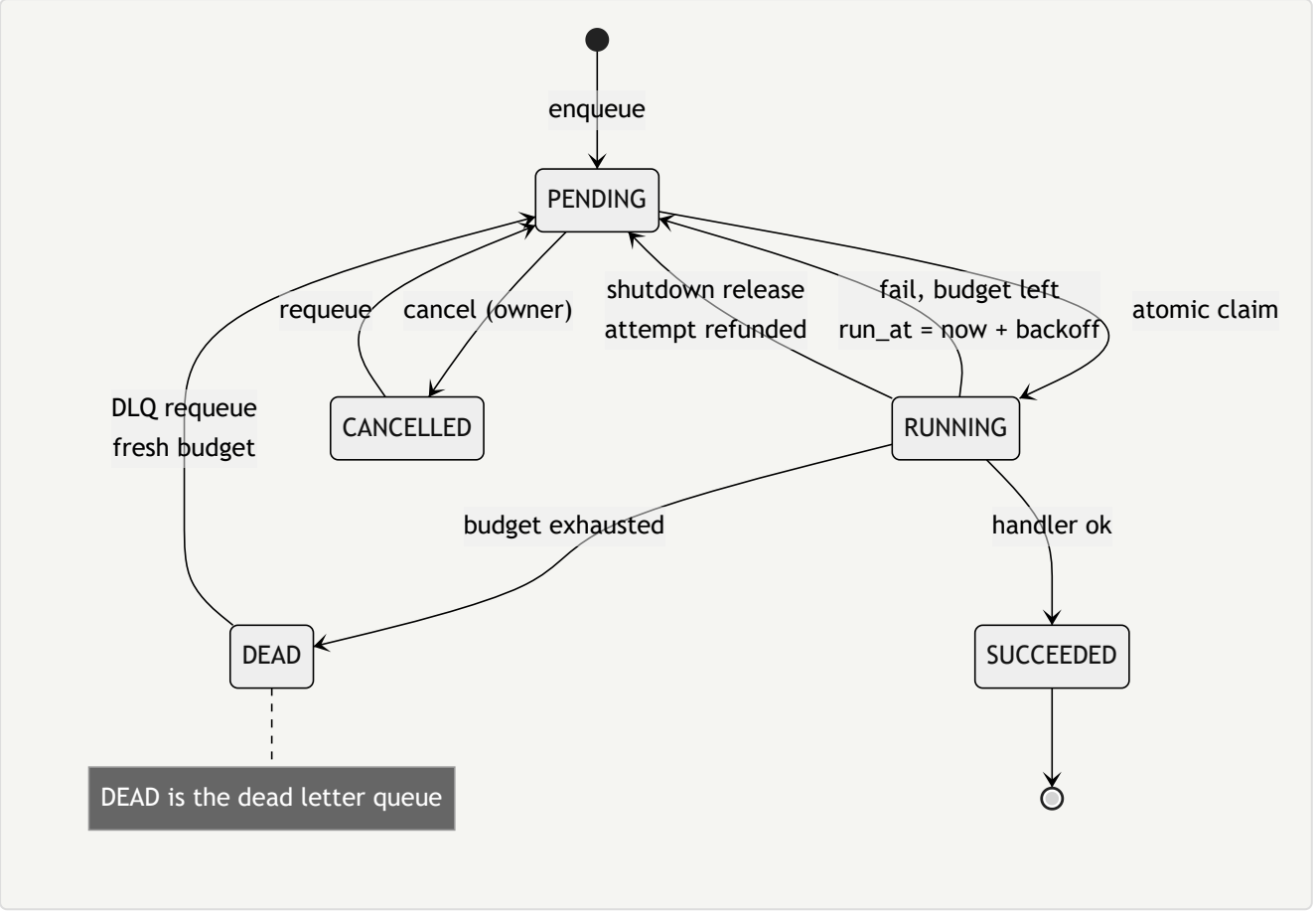
Chronos — Diagrams

Mermaid sources. Render on GitHub, or export to PDF with `mmdc -i DIAGRAMS.md -o diagrams.pdf` (mermaid-cli).

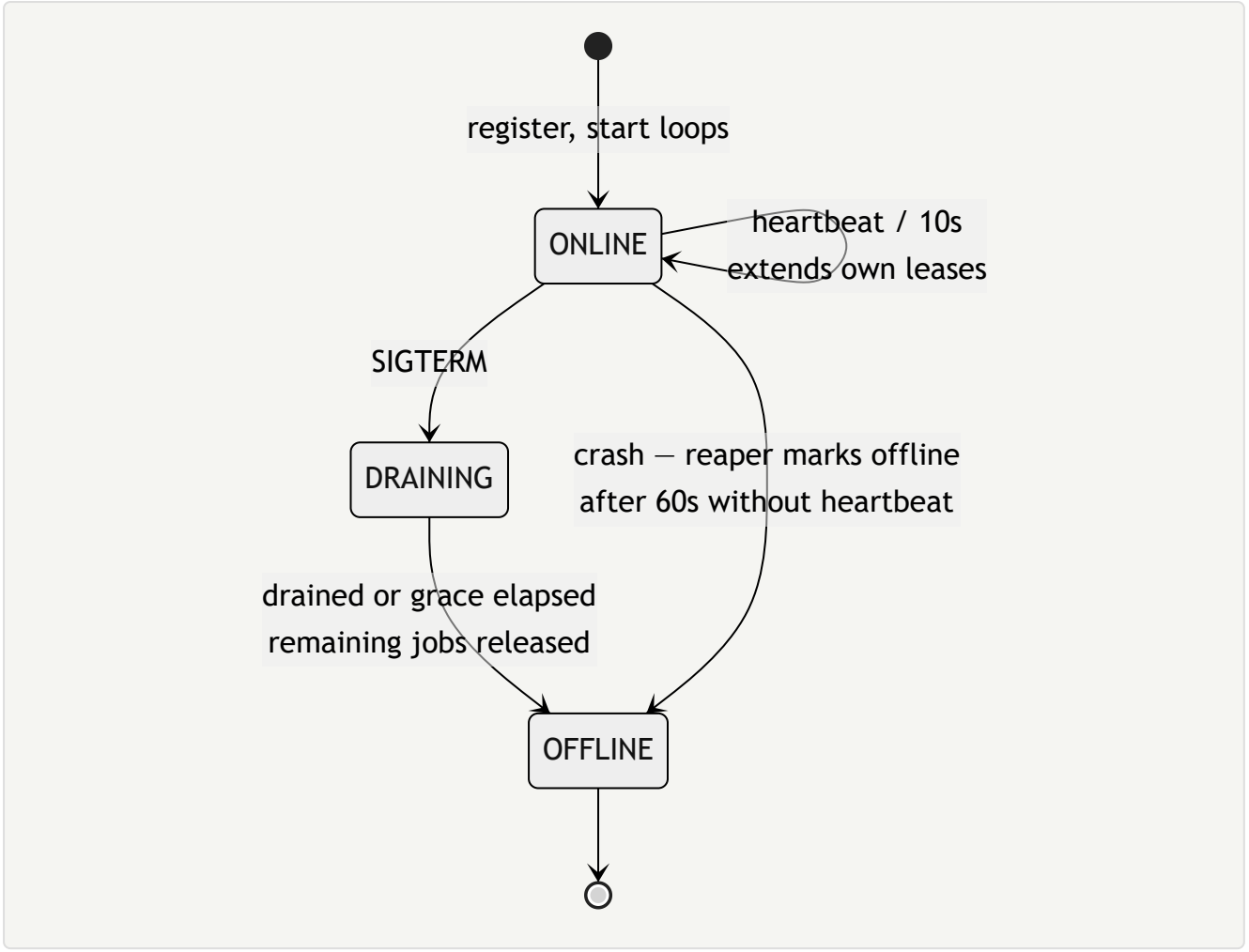
1. System architecture



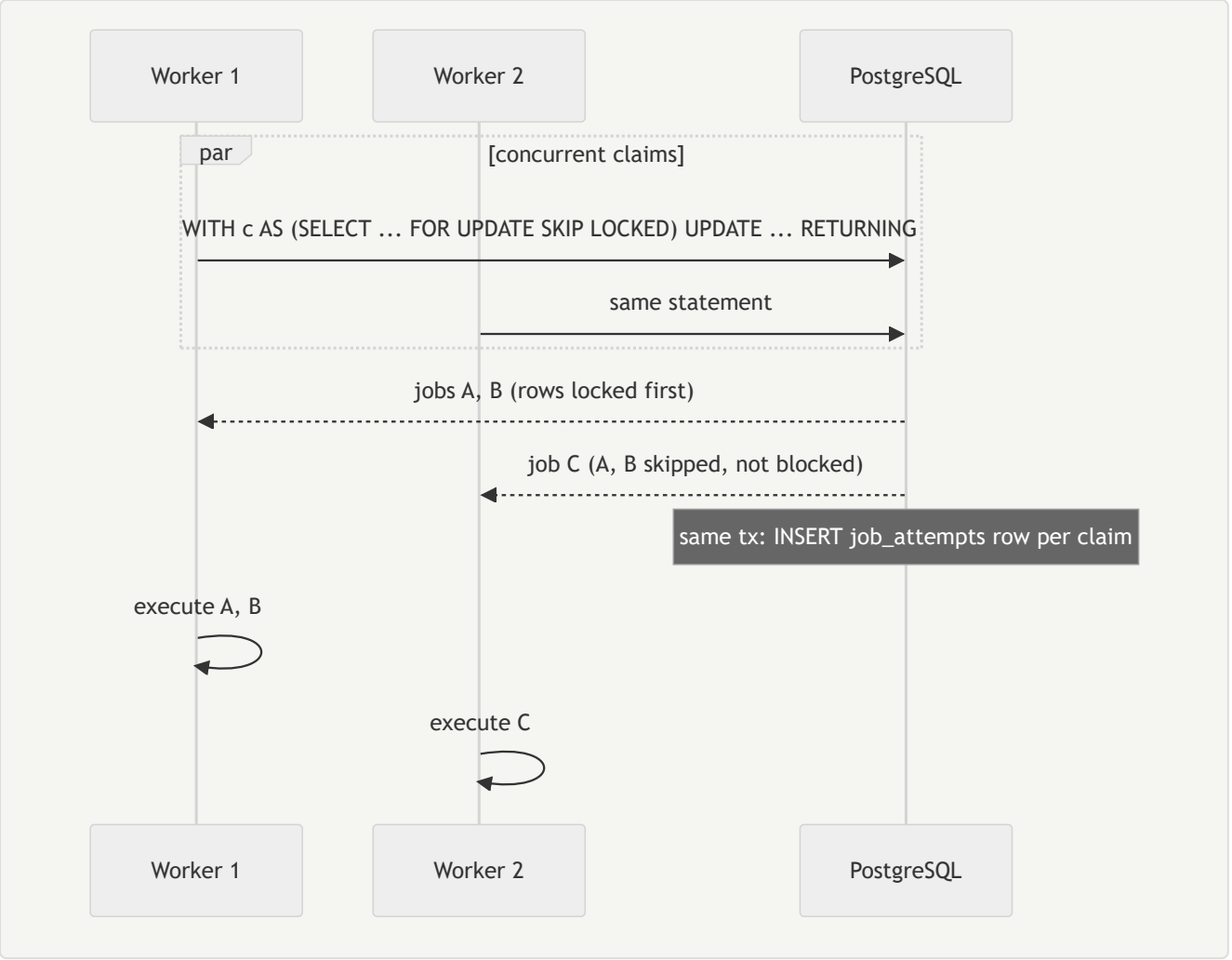
2. Job lifecycle



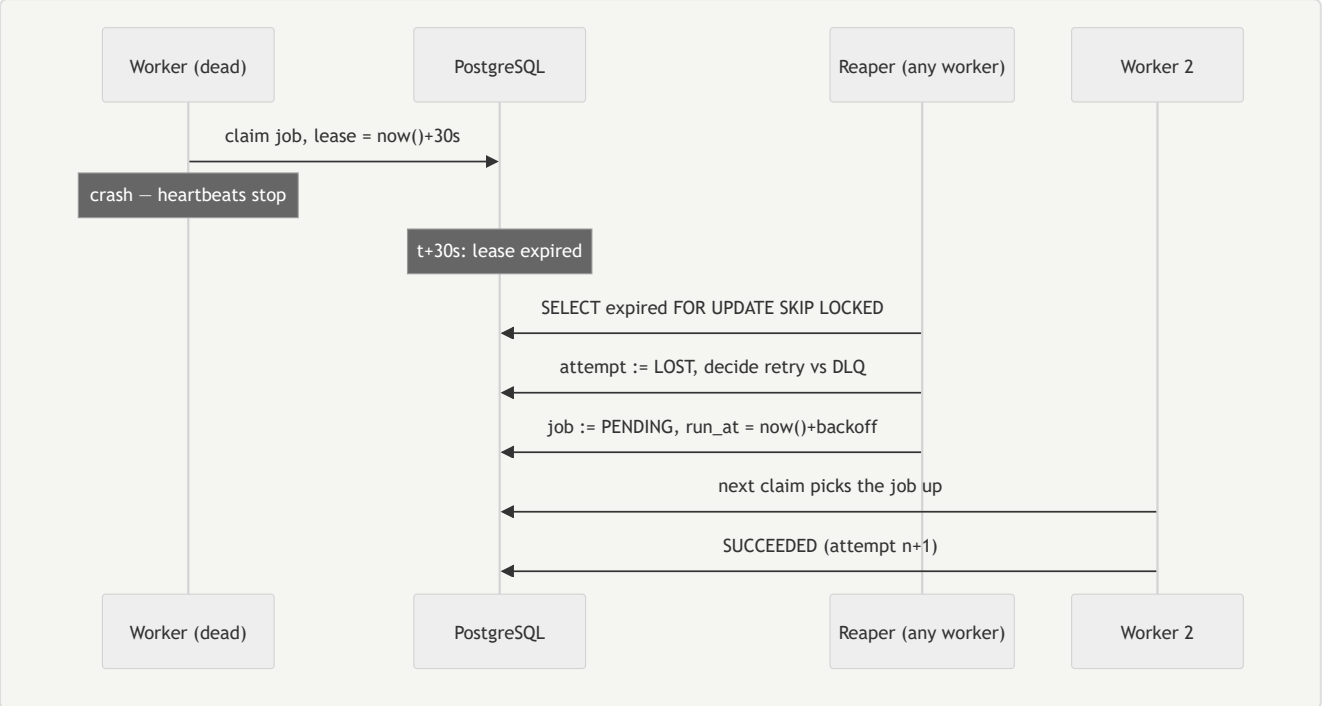
3. Worker lifecycle



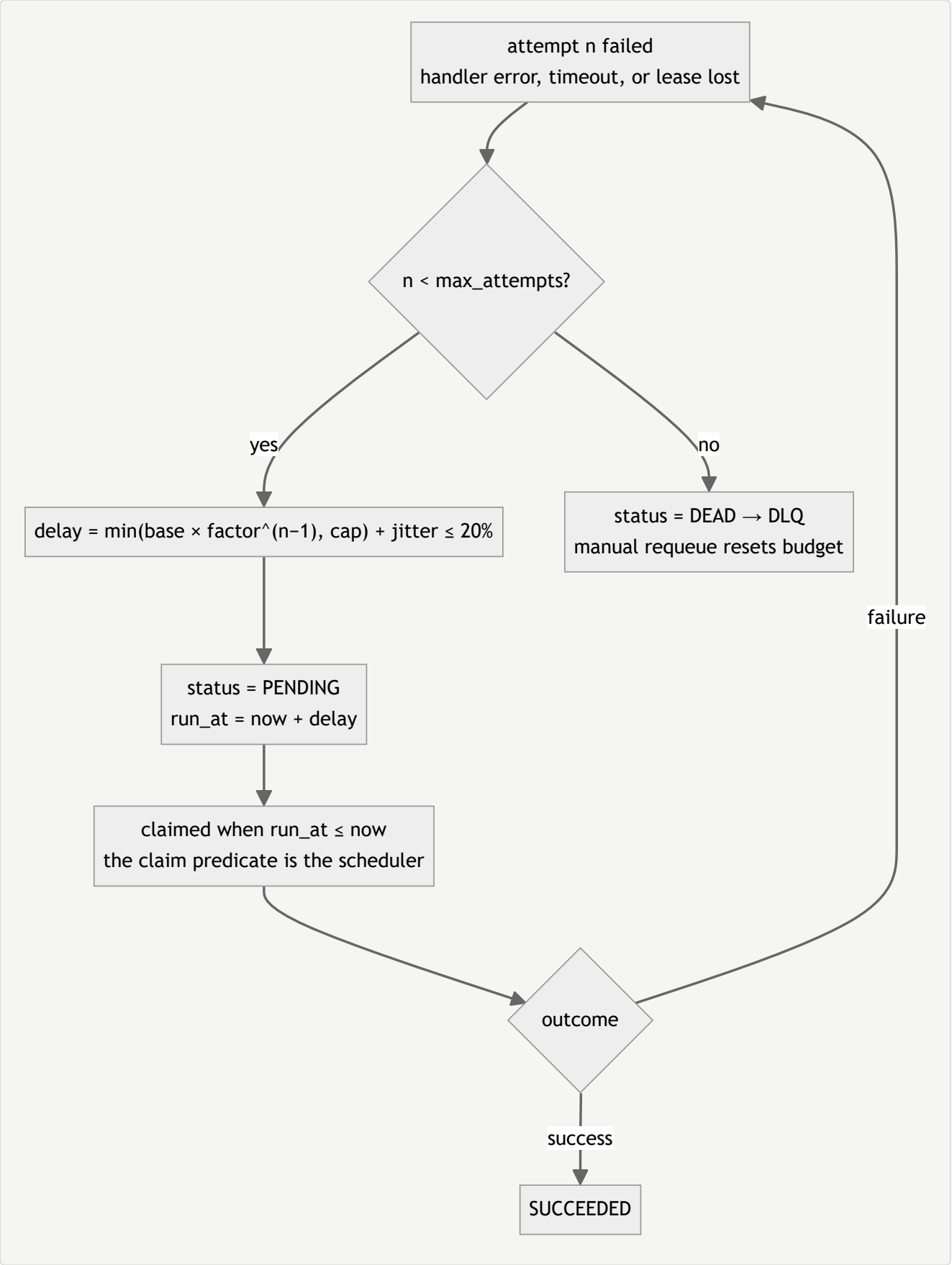
4. Claim sequence (two workers, no double-claim)



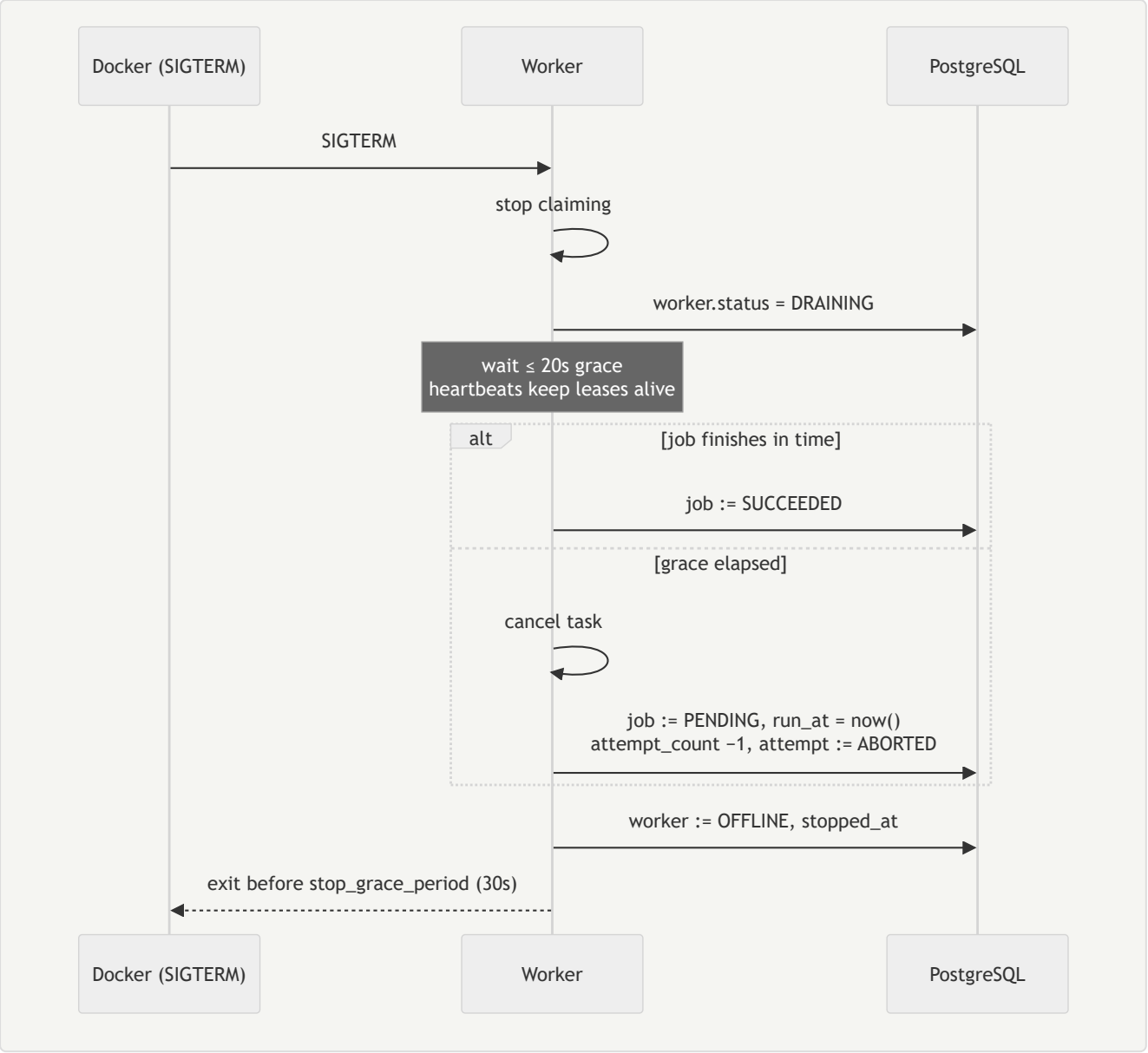
5. Lease expiry recovery



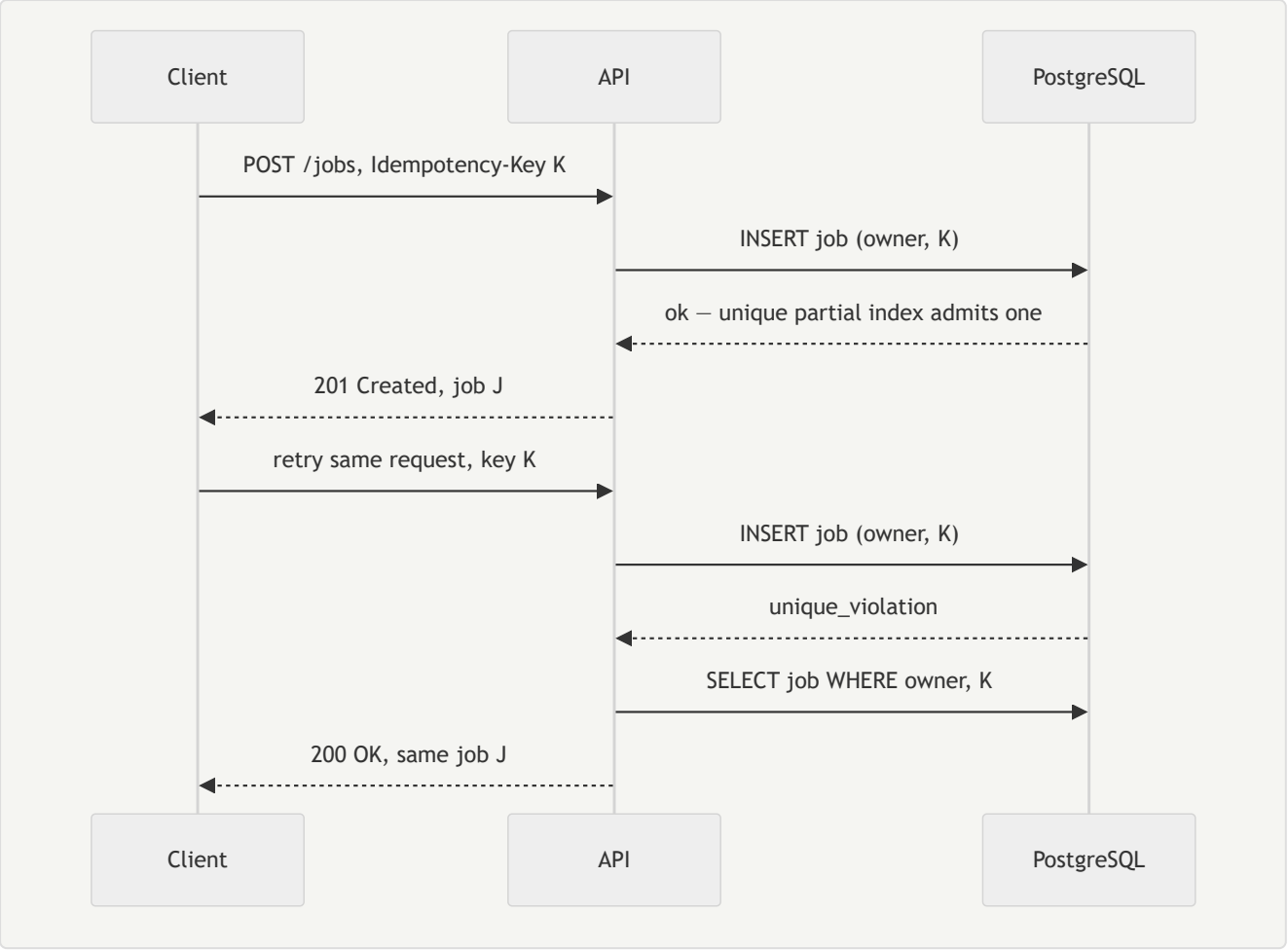
6. Retry flow


attempt n failed
handler error, timeout, or lease lost
n < max_attempts?
yes
no
delay = min(base × factor^(n−1), cap) + jitter ≤ 20%
status = DEAD → DLQ
manual requeue resets budget
failure
status = PENDING
run_at = now + delay
claimed when run_at ≤ now
the claim predicate is the scheduler
outcome
success
SUCCEEDED

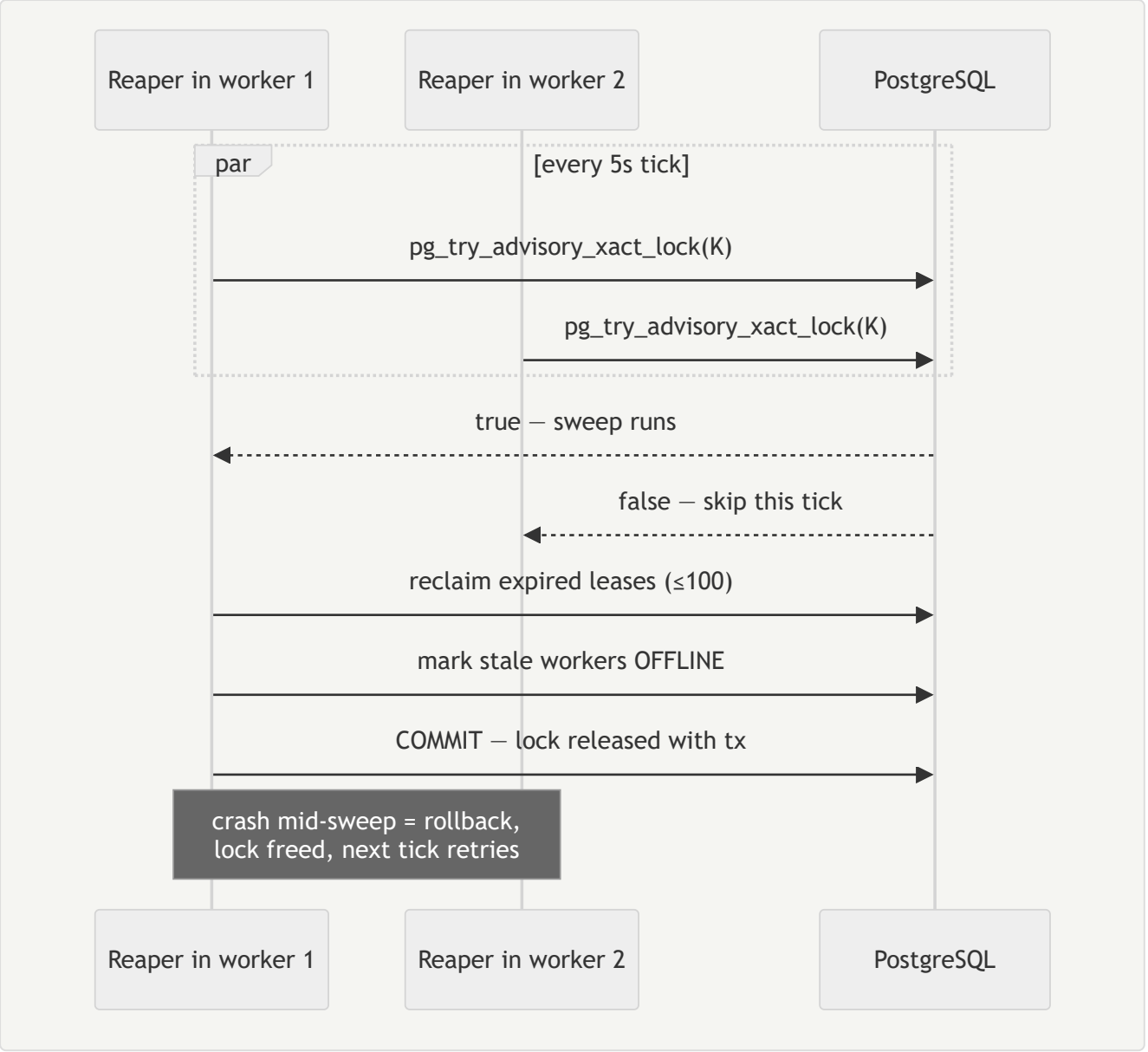
7. Graceful shutdown



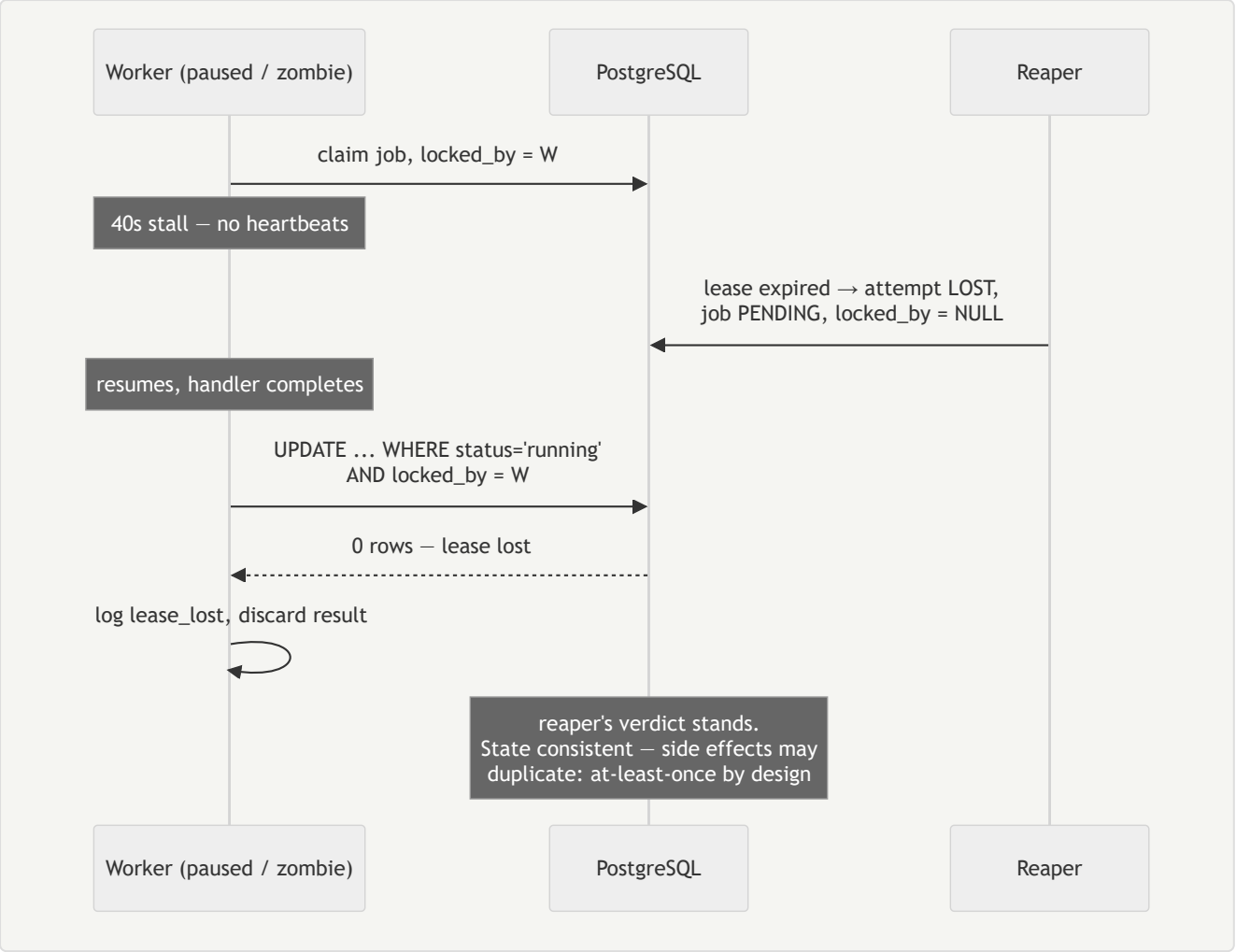
8. Idempotency flow



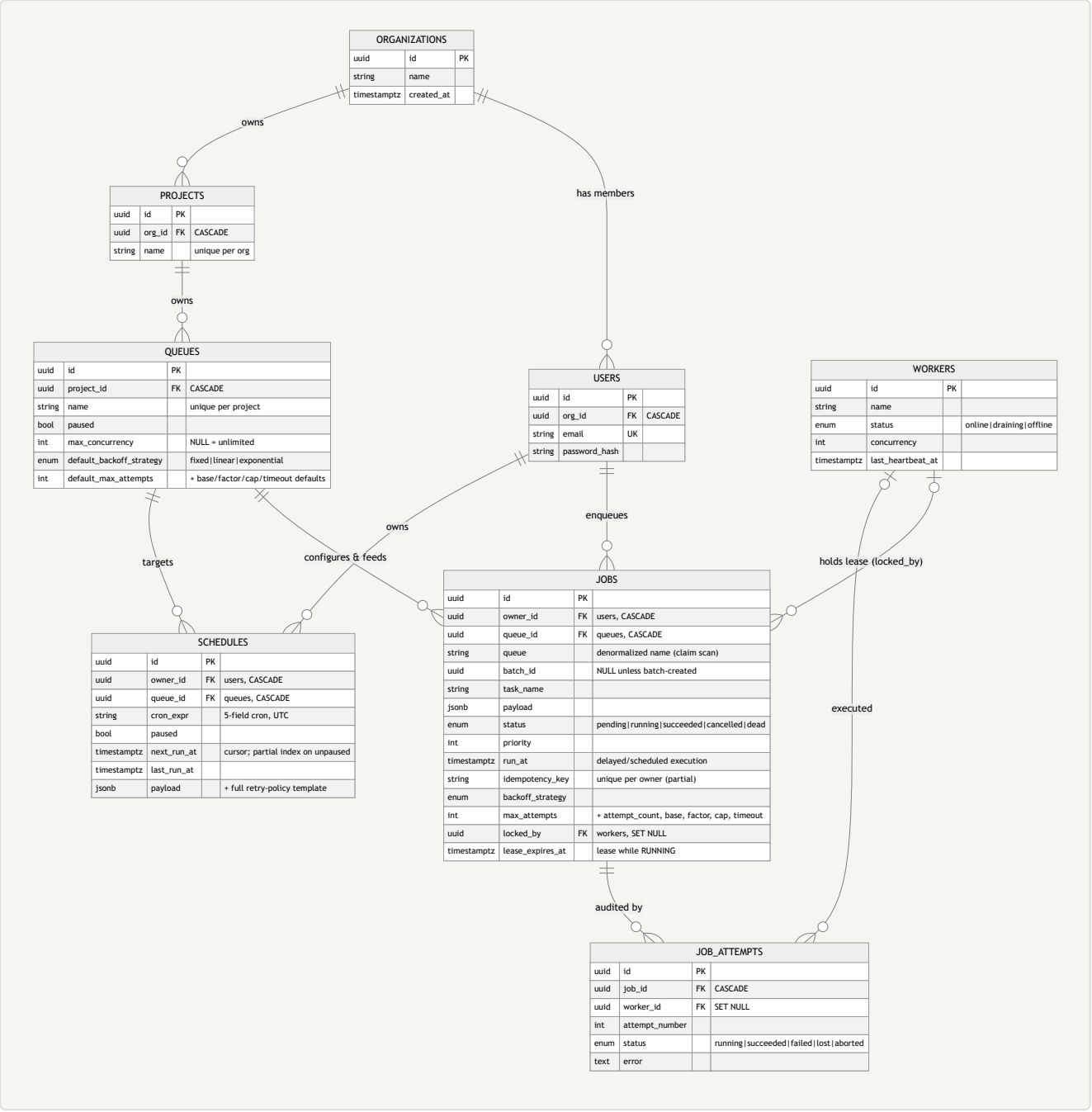
9. Reaper coordination



10. Worker crash recovery (fencing)



11. ER diagram



Key index/constraint decisions (details in DESIGN_DOC §5): the claim path uses a *partial* index on pending jobs `(priority DESC, run_at)`; leases use a partial index on running jobs; idempotency is a unique partial index on `(owner_id, idempotency_key)`; per-queue running counts use a partial index on `(queue_id) WHERE status='running'`; the DLQ is `status = 'dead'`, not a table — a dead job keeps its identity, payload and attempt history.